Determining When and What to Ask in Robot Task Execution

Changmin Park[1†], Seongah Park[1†], Nayoung Kim[1], Sungjin Park[1], Gayoung Ryu[1], Geonyeong Ko[1], Junho Lee[1*], Changjoo Nam[1,2*]

[1*]Sogang University, Seoul, Republic of Korea.
[2]Institution2, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): kjcom5322@gmail.com; cjnam@sogang.ac.kr;
Contributing authors: cmp9877@gmail.com; ppseongah@gmail.com;
nayoungkim0826@gmail.com; sungjin.park@sogang.ac.kr; gyryu.sg@gmail.com;
2567bear@naver.com;
[†]These authors contributed equally to this work.

Abstract

In this paper, we present a method for deciding when a robot should request external information and what it should ask during task execution. A robot operating with incomplete information may fail to determine the current state of the world, which can lead to task failure. However, resolving every such ambiguity through external queries would incur unnecessary costs. Therefore, a robot must determine both when to query and what information to request. To solve this problem, we maintain a weighted belief over possible task states. We use belief uncertainty to decide when to query and select the hypothesis with the highest expected uncertainty reduction to determine what to ask. After receiving an answer from an external expert, the belief is updated and task execution continues. We evaluate the framework in two robotic task domains and compare it with alternative strategies for query timing and query selection. The results show that deciding when to query has a substantial effect on task success, whereas deciding what to query mainly affects the number of required queries. Compared with conformal-prediction-based and reward-based strategies, our method achieves higher task success, highlighting the importance of query timing and selection beyond simply minimizing the number of queries.

Keywords: Planning under uncertainty, online symbolic planning, robotic systems

1 Introduction

Robots operating in real-world environments often have incomplete or unreliable information about the current task state due to perception errors Kaipa et al. (2016), execution failures Kaelbling et al. (1998), and unexpected environmental changes Kurniawati and Yadav (2016). If such uncertainty is not resolved, the robot may make incorrect decisions and fail the task. External information from an expert can help resolve this uncertainty Dragan et al. (2013); Goodrich and Schultz (2008), but frequent queries can incur substantial costs in time and workload Chen and Barnes (2014). The robot must therefore determine *when* external information is necessary and *what* information should be requested.

Recent uncertainty-aware systems selectively request external feedback during task execution.

Existing approaches commonly trigger queries based on prediction confidence, including conformal prediction Ren et al. (2023); Liang et al. (2024); Mullen and Manocha (2025), or on the expected effect of a query on task reward Burks et al. (2021); Anand et al. (2025). While these strategies reduce unnecessary interaction, they do not explicitly use uncertainty about the current task state. As a result, they may fail to identify when additional state information is needed or what information would be most useful to request. In contrast, we determine when to query based on state uncertainty and what to query based on its expected reduction.

In this paper, we present a method that maintains a weighted belief over possible task states throughout execution. We formulate decision making as a partially observable Markov decision process (POMDP) Kaelbling et al. (1998) and represent task knowledge using a STRIPS-style symbolic model Davis et al. (1993); Fikes and Nilsson (1971). The belief is updated after each action and observation, and its uncertainty is used to determine when external information is needed. If a query is triggered, the robot selects the state fact expected to reduce uncertainty the most and asks an external expert to verify it. The response is incorporated into the belief, which is then used for subsequent planning and execution.

We evaluate the method in tomato harvesting and waste sorting, which exhibit different forms of state uncertainty. Using oracle feedback, we analyze the effects of query timing and query selection and compare against conformal-prediction-based and reward-based strategies. We then evaluate non-oracle feedback from a VLM and human participants and compare against KnowNo Ren et al. (2023). Finally, interviews with five participants provide qualitative observations on the proposed querying behavior.

The main contributions of this work are as follows:

- We determine when to query based on uncertainty in the current task state and show that query timing has a substantial effect on task success.
- We prioritize state information that is expected to reduce uncertainty and show that query selection affects the number of required queries.
- We compare the proposed method with conformal-prediction-based and reward-based strategies and show that it achieves higher task success without substantially increasing the number of queries.

2 Related Work

2.1 Planning under Uncertainty

Symbolic task representations such as STRIPS and PDDL Fikes and Nilsson (1971); Ghallab et al. (1998) describe task-relevant objects, predicates, goals, and action transitions in an explicit form. In symbolic task planning, action applicability depends on the facts that hold in the current state Fikes and Nilsson (1971); Helmert (2009). Uncertainty about these facts can therefore affect which actions and plans are considered feasible. However, standard symbolic planning generally assumes a single current state and does not directly account for noisy observations, hidden information, or multiple competing interpretations of the world.

POMDPs provide a general framework for planning under such uncertainty by maintaining a belief over possible states and accounting for future observations when selecting actions Kaelbling et al. (1998); Pineau et al. (2003); Ross et al. (2008); Somani et al. (2013). Exact belief-space planning is often intractable, so approximate methods use structure or samples to represent the belief. Factored POMDP approaches exploit structure in state, observation, or value-function components Hansen and Feng (2000); Feng and Hansen (2001); Veiga et al. (2014). Related inference methods maintain factorized distributions for structured dynamic Bayesian networks Murphy and Weiss (2013). These methods improve scalability but can require assumptions or approximations about dependencies among state variables.

Particle-based approaches instead approximate a belief with samples of complete states. Each particle retains a complete state hypothesis without an explicit factorization of the state distribution. Particle representations have been used for navigation and exploration Lauri and Ritala

(2016); Kim et al. (2021), object search Xiao et al. (2019), and manipulation Pajarinen and Kyrki (2017); Pajarinen et al. (2023). Their computational cost can increase with the number of particles required to represent a large state space. In symbolic tasks, unconstrained sampling can also produce state hypotheses that are inconsistent with task dynamics. We therefore use reachable task-state particles to retain complete state hypotheses while excluding states that cannot arise under the task model.

Several works combine symbolic task knowledge with probabilistic planning. CORPP Zhang and Stone (2015), for example, constructs priors over possible worlds from commonsense knowledge and incorporates them into POMDP planning. Other approaches integrate structured task representations with belief-space decision making Sanner and Kersting (2010); Serrano et al. (2021); Li et al. (2026). These works show how symbolic structure can organize probabilistic reasoning over task states. Building on this connection, we use a symbolic task model both to define the states in the belief and to identify the state facts available for external verification.

2.2 Selective Querying under Uncertainty

System-initiated querying allows a robot to request task-relevant information from an external source when onboard observations are insufficient Ren et al. (2023). This mechanism differs from active sensing, active perception, and exploration Lauri and Ritala (2016); Kim et al. (2021). Those methods reduce uncertainty through observations generated during interaction with the environment. External querying can provide state information that onboard observations do not reveal reliably, but each query adds interaction time and workload Chen and Barnes (2014). A selective querying method must therefore determine both when external information is necessary and what information should be requested.

Prior work determines when to query using policy confidence or learned uncertainty Singi et al. (2023); Igbinedion and Karaman (2024); Mullen and Manocha (2025), trust in human input Mangalindan et al. (2025), calibrated prediction sets Ren et al. (2023); Liang et al. (2024); Zhao et al. (2025), or the expected utility of a query Burks et al. (2021); Anand et al. (2025). KnowNo, for example, requests help when a conformal prediction set contains more than one candidate action Ren et al. (2023). These criteria identify a difficult decision, estimate the value of assistance, or account for the reliability of an information source. However, ambiguity among predicted actions does not show whether missing information about the current task state caused the difficulty. We therefore determine when to query from uncertainty in an explicit belief over current task states.

The decision of what to ask is closely coupled with the decision of when to ask because the uncertainty that triggers a query also determines which information is needed. KnowNo first constructs a conformal prediction set of candidate actions and asks the user to select among them when the set contains multiple actions Ren et al. (2023). Related approaches likewise request guidance over candidate actions or plans when predictions are uncertain Liang et al. (2024); Mullen and Manocha (2025). In these methods, the candidate set determines the query target before the external response is obtained. This design can resolve uncertainty about the next action when the correct action is among the candidates, but it does not necessarily resolve uncertainty about the current task state. We therefore select a planning-relevant state fact from the current belief according to its expected reduction of uncertainty.

Prior work has treated planning under state uncertainty and selective querying as separate problems. Planning methods use state uncertainty to select robot actions, whereas selective querying methods typically use decision confidence or expected utility to request assistance. Our work occupies the intersection of these directions by grounding planning, query timing, and query selection in uncertainty about the current task state.

3 Preliminaries

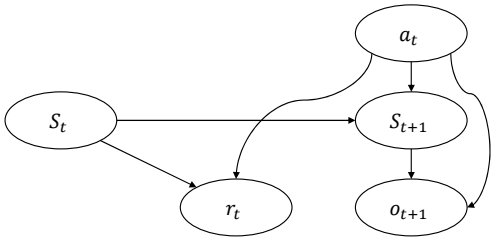


Fig. 1: Graphical illustration of a POMDP. At each time step t, the agent selects an a_t based on its current belief over the latent state s_t. The environment transitions to the next state s_{t+1} and generates a reward r_t. Since the true state is not directly observable, the agent receives an observation o_{t+1}.

3.1 Symbolic State and Action Representation

The objective of planning is to find a transition sequence that transforms the initial state into a goal state. The planning problem is defined as a tuple $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, \mathcal{G} \rangle$, where $\mathcal{F}$ is a finite set of atoms, $\mathcal{A}$ is a set of actions, $s_0 \subseteq \mathcal{F}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{F}$ is the goal condition. A state s is represented as a subset of $\mathcal{F}$. Each action $a \in \mathcal{A}$ is defined as a tuple $a = \langle \mathrm{pre}(a), \mathrm{add}(a), \mathrm{del}(a) \rangle$, where $\mathrm{pre}(a), \mathrm{add}(a), \mathrm{del}(a) \subseteq \mathcal{F}$ denote the preconditions, add lists, and delete lists of the action, respectively. An action a is said to be applicable in state s if and only if $\mathrm{pre}(a) \subseteq s$, denoted as $a[\![s]\!]$. The state transition is defined as $s \xrightarrow{a} s'$, where $s' = (s \setminus \mathrm{del}(a)) \cup \mathrm{add}(a)$.

A plan is a sequence of actions $\pi = (a_1, \ldots, a_T)$ such that the goal condition is satisfied after sequentially applying the actions starting from the initial state, i.e.,

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_T} s_T \quad \text{and} \quad \mathcal{G} \subseteq s_T.$$

3.2 POMDP

A POMDP is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and Ω is the observation space. The transition model $T(s' \mid s, a)$ specifies the probability of reaching the next state s' after taking action a in current state s. The observation model $O(o \mid s', a)$ defines the probability of receiving observation o after arriving at state s' by executing action a. The reward function $R(s, a)$ gives a scalar value to each transition, and $\gamma \in (0, 1]$ is the discount factor.

Since the true state is not directly observable, the agent maintains a belief b, which is a probability distribution over $\mathcal{S}$ such that $b(s) = \Pr(s)$. After executing an action a and receiving an observation o, the belief is updated using a Bayes filter.

$$b'(s') = \eta\, O(o \mid s', a) \sum_{s \in \mathcal{S}} P(s' \mid s, a)\, b(s),$$

where η is a normalization constant.

A policy π maps beliefs to actions. The objective of a POMDP is to find a policy that maximizes the expected cumulative discounted reward, defined as $\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$.

3.3 POMCP

Partially Observable Monte Carlo Planning (POMCP) performs Monte Carlo tree search over action-observation histories $h = \langle a_1 o_1 \ldots a_t o_t \rangle$, where each node in the search tree corresponds to a history. Action selection follows the UCB rule:

$$a^* = \arg\max_a \left(V(ha) + c \sqrt{\frac{\log N(h)}{N(ha)}} \right), \quad (1)$$

where $V(ha)$ is the estimated value of a after h, $N(h)$ and $N(ha)$ denote visit counts, and c is an exploration constant.

Each simulation samples the next state, observation, and reward from a generative model $(s', o, r) \sim \mathcal{G}(s, a)$. The return is computed recursively as $R = r + \gamma R'$, where R' is the return from subsequent steps. The value estimate is updated using an incremental average:

$$V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}.$$

The belief associated with a history h is represented implicitly as a set of particles, $B(h) = \{s \mid s \text{ is visited at } h\}$, which approximates the posterior over states without explicit belief updates.

4 Method

To determine when and what to query the user, we maintain a particle-based belief state to quantify uncertainty and identify uncertain state facts. However, particle sampling in large state spaces is computationally expensive, and unconstrained sampling may produce states that violate symbolic constraints. To address these challenges, we propose a frontier-based particle sampling approach that uses symbolic action models to maintain a constraint-consistent belief for query selection.

4.1 Planning and Querying

We maintain a knowledge state s_t^K for symbolic planning and execution. For online action selection, we employ POMCP Silver and Veness (2010), restricting tree expansion and rollout to symbolically applicable actions. At each step, the planner selects an action $a_t \in \mathcal{A}$ applicable to s_t^K, denoted by $a_t[\![s_t^K]\!]$. Following execution and observation o_{t+1}, we construct a weighted belief $\mathcal{B}_{t+1}$ by sampling from the reachable-state frontier defined by (s_t^K, a_t). This restricts belief construction to symbolically valid successors while capturing uncertainty in the action outcome.

4.1.1 When to Query

To determine when to query, we assess uncertainty about the current state through the entropy of the weighted belief. If several successor states remain plausible after an action and no single state is dominant, the robot cannot confidently identify its current state. In this situation, belief entropy is high because the weight is distributed across several possible states. We use this property of entropy to compute confidence from normalized belief entropy Qu et al. (2023), where lower confidence indicates greater state uncertainty.

We observe that a symbolic action usually changes only part of the state. Based on this observation, we define the *frontier* of reachable successor states as follows.

$$\mathcal{F}_{t+1} = \text{Frontier}(s_t^K, a_t) = \{s_{t+1}^1, \ldots, s_{t+1}^j\}. \quad (2)$$

For example, an action to pick a tomato can change which object the robot holds but preserves the robot location. Uncertain outcomes within the frontier are mutually exclusive, and no two successor states are identical. We restrict particle sampling to this frontier instead of the entire symbolic state space.

We construct the weighted belief over the j sampled frontier states with normalized weights w_k.

$$\mathcal{B}_{t+1} = \{(s_{t+1}^k, w_k)\}_{k=1}^j, \qquad \sum_{k=1}^{j} w_k = 1. \quad (3)$$

Specifically, we compute these weights by normalizing the product of the transition probability from s_t^K and the observation likelihood.

$$\begin{aligned} \bar{w}_k &= T(s_{t+1}^k \mid s_t^K, a_t)\, O(o_{t+1} \mid s_{t+1}^k, a_t), \\ w_k &= \frac{\bar{w}_k}{\sum_{\ell=1}^{j} \bar{w}_\ell}. \end{aligned} \quad (4)$$

If all $\bar{w}_k$ are zero, we set $w_k = 1/j$.

This explicit weighted frontier is used for confidence estimation and query selection. POMCP maintains sampled states in the search tree to estimate action values. Both components use the same symbolic transition and observation models.

We compute the entropy of the belief as

$$\mathcal{H}(\mathcal{B}_{t+1}) = -\sum_{k=1}^{j} w_k \log_2 w_k.$$

Because the maximum entropy depends on the number of represented worlds, we normalize by $\log_2 j$ and define

$$\text{Conf}(\mathcal{B}_{t+1}) = 1 - \frac{\mathcal{H}(\mathcal{B}_{t+1})}{\log_2 j}. \quad (5)$$

For $j = 1$, we define $\text{Conf}(\mathcal{B}_{t+1}) = 1$. The score approaches zero if the represented states have similar weights and approaches one if most of the belief weight lies on one state.

Given a predefined threshold $\tau \in [0, 1]$, the robot triggers a query if

$$\text{Conf}(\mathcal{B}_{t+1}) < \tau. \quad (6)$$

A higher threshold τ requires a more concentrated belief and can increase the number of queries.

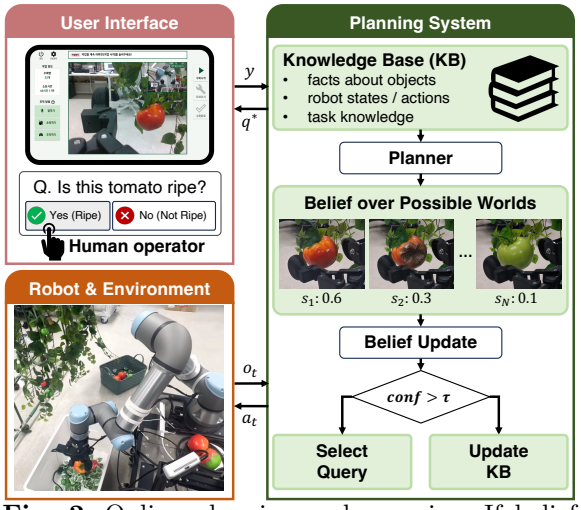


Fig. 2: Online planning and querying. If belief confidence falls below τ, the robot queries the selected fact q^* and uses the Boolean response $y \in \{\texttt{true}, \texttt{false}\}$ to update the belief and knowledge base.

4.1.2 What to Query

If belief confidence falls below τ, the robot triggers a query and must decide which fact to verify. Since verification eliminates state hypotheses that are inconsistent with the answer, we select the fact with the largest expected reduction in belief entropy.

Specifically, we define the query candidate set $\mathcal{Q}(\mathcal{B})$ by extracting atomic facts whose truth values differ across the remaining frontier states in the belief,

$$\mathcal{Q}(\mathcal{B}) = \bigcup_{(s,w)\in\mathcal{B}} s \setminus \bigcap_{(s,w)\in\mathcal{B}} s.$$

For each candidate fact $q \in \mathcal{Q}(\mathcal{B})$, we partition the belief according to whether q holds in each state,

$$\mathcal{B}^{q+} = \{(s^k, w_k) \mid q \in s^k\},$$
$$\mathcal{B}^{q-} = \{(s^k, w_k) \mid q \notin s^k\}.$$

Let $P(q) = \sum_{k:q\in s^k} w_k$ and $P(\neg q) = 1 - P(q)$. Entropy within each subset is computed after conditionally normalizing its weights. The expected posterior entropy of asking about q is

$$\mathbb{E}[\mathcal{H} \mid q] = P(q)\mathcal{H}(\mathcal{B}^{q+}) + P(\neg q)\mathcal{H}(\mathcal{B}^{q-}).$$

Algorithm 1: Planning and Querying

Input initial state s_0^K, goal condition $\mathcal{G}$, confidence threshold τ, step limit $T_{\max}$
Output executed action sequence π

$\mathcal{B} \leftarrow \{(s_0^K, 1)\}$ $\quad t \leftarrow 0$ $\quad \pi \leftarrow ()$
while $t < T_{\max} \wedge \mathcal{G} \not\subseteq s_t^K$ **do**
  $a_t \leftarrow$ `SEARCH`$(\mathcal{B})$
  if $a_t = \emptyset$ **then**
    break `// dead end`
  execute a_t and receive o_{t+1}
  $\pi \leftarrow \pi \,\|\, a_t$
  $\mathcal{B} \leftarrow$ `UPDATE-BELIEF`(s_t^K, a_t, o_{t+1})
  $c \leftarrow \mathrm{Conf}(\mathcal{B})$
  while $c < \tau$ **do**
    $q^* \leftarrow$ `SELECT-FACT`$(\mathcal{B})$
    if $q^* = \emptyset$ **then**
      break `// no differentiating fact`
    $y \leftarrow$ `QUERY`(q^*)
    $\mathcal{B} \leftarrow$ `FILTER`$(\mathcal{B}, q^*, y)$
    $c \leftarrow \mathrm{Conf}(\mathcal{B})$
  $s_{t+1}^K \leftarrow \arg\max_{s^k\in\mathcal{B}} w_k$
  $t \leftarrow t + 1$
return π

The selected query is

$$q^* = \arg\min_{q\in\mathcal{Q}(\mathcal{B})} \left[P(q)\mathcal{H}(\mathcal{B}^{q+}) + P(\neg q)\mathcal{H}(\mathcal{B}^{q-})\right]. \tag{7}$$

Eq. 7 selects the fact q^* that maximizes expected entropy reduction. The robot then converts this fact into a binary query about the current state. For example, $q^* = \texttt{ripe}(\text{tomato}_1)$ produces the question *'Is tomato 1 ripe?'*. Based on the response $y \in \{\texttt{true}, \texttt{false}\}$, the robot updates the belief by removing states inconsistent with the reported truth value,

$$\mathcal{B} \leftarrow \begin{cases} \{(s^k, w_k) \mid q^* \in s^k\}, & y = \texttt{true}, \\ \{(s^k, w_k) \mid q^* \notin s^k\}, & y = \texttt{false}. \end{cases} \tag{8}$$

The retained weights are renormalized, confidence is recomputed, and another fact is selected if confidence remains below τ. The loop ends if the threshold is met or no differentiating fact remains. The robot then commits the maximum a

posteriori state

$$s_{t+1}^K = \arg \max_{s^k \in \mathcal{B}_{t+1}} w_k$$

for subsequent symbolic planning.

Alg. 1 explains the overall online planning and querying procedure. The robot first initializes the belief, step counter, and action sequence (line 3). Execution continues while $t < T_{\max}$ and the goal condition is not satisfied, $\mathcal{G} \not\subseteq s_t^K$ (line 4). At each step, POMCP selects an action (line 5), and execution stops if no action is available (line 7). Otherwise, the robot executes the action (lines 8 and 9), then constructs the weighted frontier belief and computes confidence (lines 10 and 11). If confidence falls below τ, the robot selects a candidate fact (line 13). If no candidate remains, the query loop ends (line 15). Otherwise, the robot obtains an external response and filters the belief (lines 16 and 17), then recomputes confidence (line 18). After the query loop, the most probable remaining state becomes the next symbolic planning state (line 19). Appx. A provides the detailed POMCP search, simulation, and rollout procedures.

4.2 System Implementation

We implement a modular framework to execute the proposed planning and querying method on a robot, as shown in Fig. 3. The Knowledge Base (KB) stores symbolic facts and action models, and Sensing and Perception (SP) provides observations and numeric state estimates from sensor data. The Planning Manager (PM) uses this information to select robot actions, construct the frontier belief, and decide when and what to query. The Feedback Manager (FM) converts the selected fact into a question and the response into evidence for the belief update. The User Interface (UI) presents the questions and collects responses during human interaction.

The KB partitions information into domain knowledge $\mathcal{I}^D$, static structured information $\mathcal{I}^S$, and local information $\mathcal{I}^L$ obtained during execution.

$$\mathcal{KB} = \mathcal{I}^D \sqcup \mathcal{I}^S \sqcup \mathcal{I}^L.$$

Domain knowledge contains action schemas and task rules. Static information includes invariant facts and fixed map or robot parameters. Local information contains perceptual facts and numeric fluents that are updated during execution. SP maps raw sensor input to these task-level observations. For example, RGB-D sensing can update object pose and ripeness estimates, while the symbolic observation likelihood determines how those observations weight the frontier states.

Each symbolic action invokes a robot skill. Actions such as `navigate`(robot, l_1, l_2) and `pick`$(\text{robot}, \text{tomato}_1)$ specify task-level intentions, while an off-the-shelf motion planner such as MoveIt Chitta et al. (2012) uses numeric fluents from the KB to generate low-level motion.

The FM maps the selected predicate to a predefined question template and converts the response to a Boolean value. For example, the predicate $\text{ripe}(t_1)$ is converted into the question "Is tomato t_1 ripe?" When interacting with human participants, the tablet UI displays the camera view, highlights the referenced object with a bounding box, and presents simple response options. Participants answer the question based on the displayed information, and the planner updates its belief and selects the next robot action accordingly.

5 Experiments

We evaluate our method in simulation and on physical robots. The evaluation covers WasteSorting and TomatoHarvesting. We first compare the complete query policies in simulation with correct oracle responses. We then evaluate physical robot execution with responses from a vision-language model (VLM) and human participants. Ablation experiments examine the contributions of query timing and query selection. A threshold sweep examines the balance between task success and query count.

The oracle experiments include 200 trials per domain and condition. The VLM experiments include 40 trials per domain and method. The planned human evaluation is designed for five participants with 10 trials per domain and method for a total of 40 trials.

5.1 Experimental Setup

The oracle experiments use simulated task execution and exact external responses. The VLM and human experiments use physical robots in

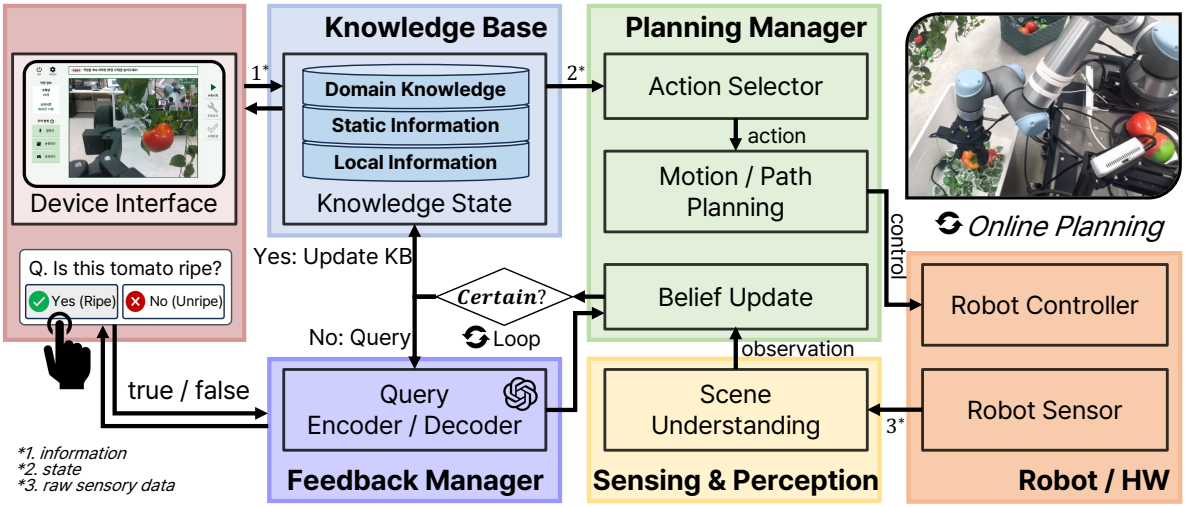


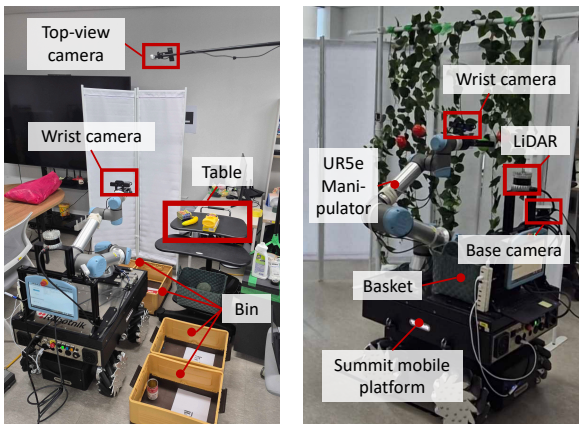
Fig. 3: Framework for robot execution of the proposed planning and querying method. The Knowledge Base (KB) stores shared symbolic facts and action models, while Sensing and Perception (SP) supplies observations from robot sensors. The Planning Manager (PM) selects actions and determines queries. If belief confidence falls below τ, the Feedback Manager (FM) converts the selected fact into a question that the User Interface (UI) presents to the human. The FM converts the response into Boolean evidence, which the PM uses to update the belief and continue planning.

laboratory environments. Fig. 4 shows the physical setups. *WasteSorting* uses a fixed UR5e robot arm. *TomatoHarvesting* uses a Summit mobile platform equipped with a UR5e robot arm. The mobile platform uses an RGB-D camera for perception and LiDAR for localization. WasteSorting uses SAM3 for object detection Carion et al. (2025). TomatoHarvesting uses YOLOv12 for object detection Tian et al. (2025).

5.1.1 Task Domains

The WasteSorting task requires the robot to place every object in the correct bin. The UR5e robot is positioned between the objects and recycling bins in Fig. 4a. All objects are within reach. The robot can detect objects and then pick and place them. Each object belongs to one of four waste categories corresponding to general waste or recyclable paper or cans or plastic. The categories are initially unknown. The main sources of uncertainty in this domain are occlusion and noisy sensor observations. In simulation we randomize the initial object arrangement for each trial. A trial succeeds if the robot places all objects in the correct bins within the step limit. Each correct placement yields a task reward of +5. Completion of the task yields an additional reward of +10. A penalty of −10 applies to the third and each subsequent detection action in a consecutive sequence.

The TomatoHarvesting task requires the robot to harvest fresh ripe tomatoes and discard rotten tomatoes. Four tomatoes are distributed across two stems in Fig. 4b. The robot navigates between the docking station and the two stems. The `detect` action distinguishes ripe from unripe tomatoes. After a tomato is picked the `scan` action distinguishes fresh from rotten tomatoes. The robot uses `place` for fresh tomatoes and `discard` for rotten tomatoes. Unripe tomatoes remain on the stem. The main source of uncertainty in this domain is noisy sensor observations. In simulation we randomize tomato locations across the two stems for each trial. A trial succeeds if all fresh ripe tomatoes are harvested and all rotten tomatoes are discarded within the step limit. Correct placement of a fresh tomato or disposal of a rotten tomato yields a task reward of +10. Consecutive `detect` actions at the same stem and consecutive `scan` actions on the same tomato incur a penalty of −5 from the second action onward.



(a) WasteSorting (b) TomatoHarvesting

Fig. 4: Physical robot setups for the two task domains. (a) WasteSorting uses a fixed UR5e arm to sort objects into the appropriate bins. (b) TomatoHarvesting uses a Summit mobile platform with a UR5e arm to navigate between tomato stems and harvest tomatoes. Annotations identify robot components, sensors, and task equipment.

5.1.2 Implementation Details

Our POMCP planner uses 100 simulations per planning step with a maximum simulation depth of 20. The discount factor is $\gamma = 0.2$ and the UCB exploration constant is $c = 1.0$. The default confidence threshold is $\tau = 0.8$. We use Gemma 4 31B IT with NVFP4 quantization Gemma Team (2026) as the external expert in the VLM experiments. Appendix C provides the baseline configurations and additional experimental details.

5.1.3 Evaluation Metrics

Success rate is the fraction of all trials that achieve the task goal within the trial limit. Query count is the number of external requests per trial. Query probability is the fraction of physical steps that trigger at least one query. Plan length is the number of physical actions executed per trial. Search time is the cumulative time spent on plan search per trial. Interaction time is the cumulative time spent on external requests and response processing per trial. We report interaction time only for the VLM and human experiments. We note that all metrics except success rate use successful trials.

5.2 System Evaluation

Table 1 summarizes the baseline and ablation configurations. The baseline comparison in Sec. 5.2.1 evaluates KnowNo, IntroPlan, and Query-Action against Ours. The baseline evaluation covers oracle responses in simulation and VLM and human responses during physical execution. The ablation study in Sec. 5.2.2 evaluates Fully-Random, Q1–Q2, and W1–W3 against Ours where W1–W3 examine query timing (when) and Q1–Q2 examine query selection (what). Finally the threshold study in Sec. 5.2.3 examines how the confidence threshold affects task success and query count. Both the ablation and threshold studies use oracle responses in simulation.

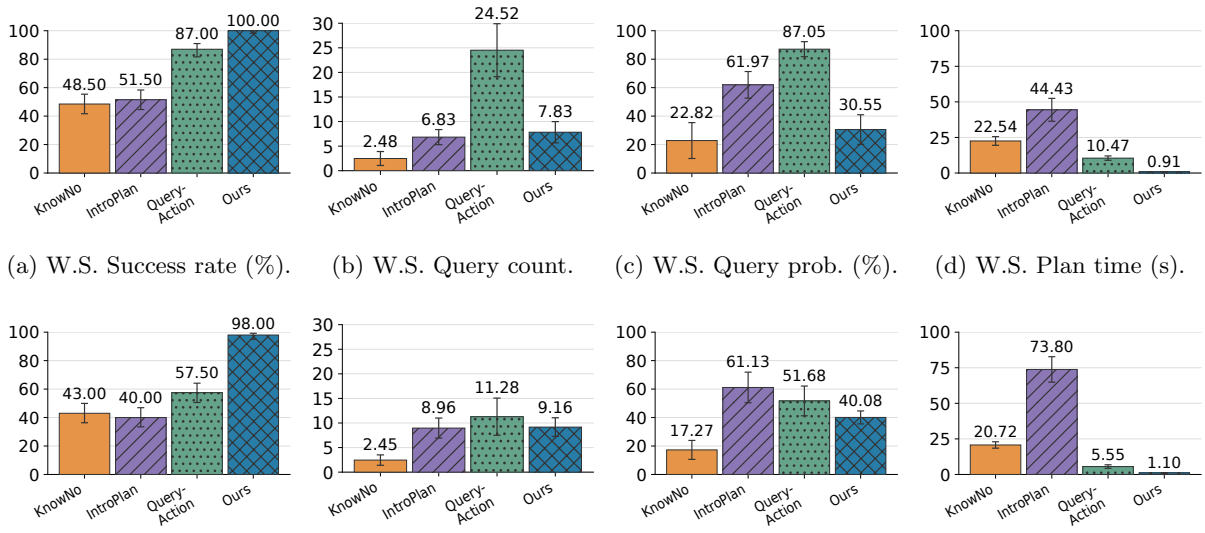
5.2.1 Baseline Comparison

Oracle. The oracle comparison includes three baselines with the query timing and selection strategies summarized in Table 1. KnowNo Ren et al. (2023) requests an action choice if its conformal prediction set does not specify a single ordinary action. IntroPlan Liang et al. (2024) extends KnowNo and incorporates retrieved examples and explanations into candidate evaluation before it requests an action choice. Query-Action follows the standard POMDP approach to information gathering Kaelbling et al. (1998). It includes Boolean state queries in the POMCP action space and selects them according to expected cumulative reward. Table 2 and Fig. 5 compare the four methods.

As shown in Table 2 and Fig. 5, our method achieves the highest success rate in both domains. It completes all 200 WasteSorting trials and 196 of 200 TomatoHarvesting trials. Our method achieves success rates of 100.00% in WasteSorting and 98.00% in TomatoHarvesting compared with 87.00% and 57.50% for Query-Action. The mean query counts are also lower at 7.83 and 9.16 compared with 24.52 and 11.28 for Query-Action. Two factors may explain the lower success rate and higher query count of Query-Action. First, query actions expand the action space so POMCP must evaluate more alternatives within the same simulation budget. Second, expected task reward does not explicitly measure confidence in the current

Table 1: Query timing and query selection strategies. Columns indicate query timing criteria and rows indicate query selection criteria. KnowNo and IntroPlan and Query-Action are the baselines. Fully-Random and Q1–Q2 and W1–W3 are the ablation configurations. Ours serves as the reference method for both comparisons. A dash denotes a combination not evaluated in this study.

What \ When	Random	Conformal Prediction	Reward-based	Confidence
Random	Fully-Random	–	–	Q1
Conformal Set	–	KnowNo , IntroPlan	–	–
Reward-based	–	–	Query-Action	Q2
Entropy Gain	W1	W2	W3	**Ours**



(a) W.S. Success rate (%). (b) W.S. Query count. (c) W.S. Query prob. (%). (d) W.S. Plan time (s).

(e) T.H. Success rate (%). (f) T.H. Query count. (g) T.H. Query prob. (%). (h) T.H. Plan time (s).

Fig. 5: Oracle baseline comparison. (a)–(d) show success rate, query count, query probability, and plan time for WasteSorting (short for W.S.). (e)–(h) show the same metrics for TomatoHarvesting (short for T.H.) in the same order. Across both domains, our method achieves the highest success rate and the lowest plan time. These results suggest that effective query timing and selection are more important than query count alone for reliable task execution.

state. Our method selects queries at the observation stage and restricts POMCP planning to physical actions. These results show that a higher query count alone does not guarantee a higher success rate.

Compared to KnowNo and IntroPlan, our method uses more queries. Their mean query counts are 2.48 and 6.83 in WasteSorting compared with 7.83 for our method. In TomatoHarvesting, the corresponding counts are 2.45 and 8.96 compared with 9.16 for our method. This difference in query count arises from how each method handles a query trigger. If a query is triggered, our method repeats queries until belief confidence reaches τ. KnowNo and IntroPlan ask only one question per trigger. Therefore our method can use more queries despite a lower query probability than IntroPlan. However KnowNo and IntroPlan achieve success rates of only 48.50% and 51.50% in WasteSorting and 43.00% and 40.00% in TomatoHarvesting. Therefore a system is not necessarily better simply because it uses fewer queries.

Table 2: Oracle baseline comparison results by domain. Bold values indicate the highest success rate and the lowest query count, query probability, and plan length within each domain.

Domain	Method	Success (%)	Query count	Query prob.(%)	Plan length
WasteSorting	KnowNo	48.50	**2.48 ± 1.42**	**22.82 ± 12.61**	10.78 ± 1.35
	IntroPlan	51.50	6.83 ± 1.53	61.97 ± 9.35	10.99 ± 1.49
	Query-Action	87.00	24.52 ± 5.39	87.05 ± 5.28	11.89 ± 1.53
	Ours	**100.00**	7.83 ± 2.16	30.55 ± 10.41	**10.35 ± 0.88**
TomatoHarvesting	KnowNo	43.00	**2.45 ± 1.06**	**17.27 ± 6.67**	14.01 ± 1.07
	IntroPlan	40.00	8.96 ± 2.04	61.13 ± 10.77	14.61 ± 1.46
	Query-Action	57.50	11.28 ± 3.78	51.68 ± 10.45	14.94 ± 1.70
	Ours	**98.00**	9.16 ± 1.90	40.08 ± 4.56	**13.97 ± 1.15**

Plan time is also lowest for Ours in both domains despite the additional queries compared with KnowNo and IntroPlan. In WasteSorting, mean plan time is 0.91 seconds for Ours compared with 22.54 seconds for KnowNo, 44.43 seconds for IntroPlan, and 10.47 seconds for Query-Action. In TomatoHarvesting, the corresponding values are 1.10, 20.72, 73.80, and 5.55 seconds. KnowNo and IntroPlan require language-model inference for candidate generation and scoring at each planning step. IntroPlan performs additional reasoning and candidate evaluation, which increases inference time compared with KnowNo. Ours evaluates query timing and selection from the belief state and searches only over physical actions. The smaller search space supports faster planning than Query-Action. The time comparisons suggest that belief-based search can reduce planning time without repeated language-model inference.

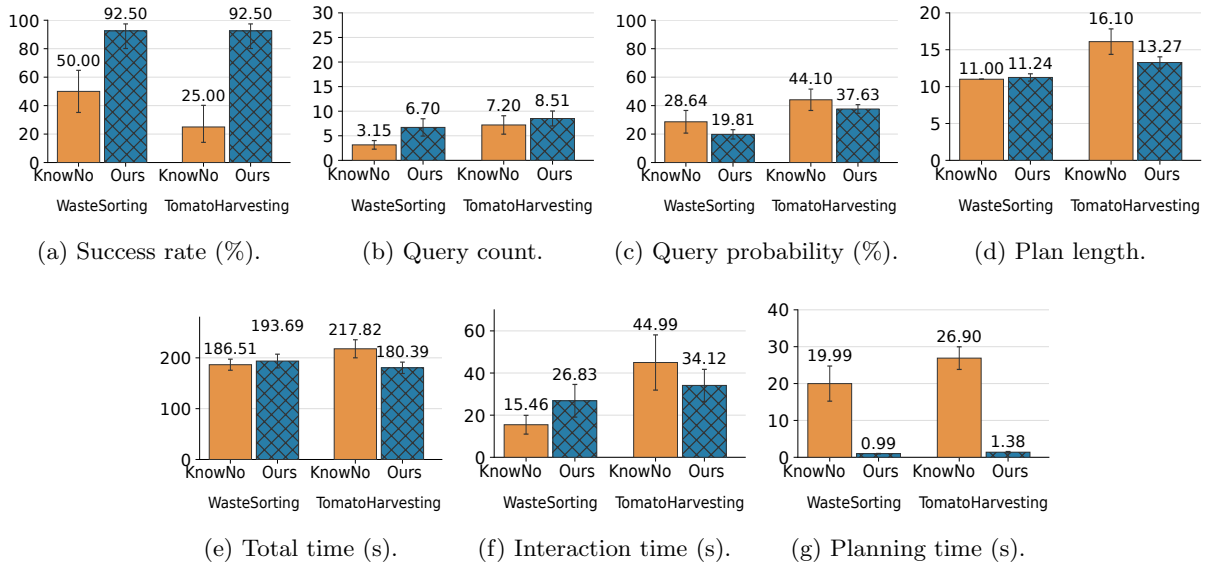
Our method combines confidence-based query timing with entropy gain query selection and achieves the highest success rate in both domains. The baseline comparisons suggest that effective query timing and selection are more important than query count alone.

VLM. We evaluate whether the trends in task success and query behavior from the oracle experiments also hold in a real-world setting. We first prompt a VLM to act as an external expert and evaluate query policies on physical robots.[1] We select KnowNo as the baseline because it achieves a high success rate relative to its query count in the oracle experiments. Each method has 40 trials per domain. Fig. 6 presents the VLM results.

[1] Appendix C.8 provides the VLM prompts.

Ours achieves a success rate of 92.50% in both domains with VLM responses. KnowNo achieves 50.00% in WasteSorting and 25.00% in TomatoHarvesting. Mean query counts are 6.70 for Ours and 3.15 for KnowNo in WasteSorting and 8.51 and 7.20 in TomatoHarvesting. Consistent with the oracle experiments, Ours requires less search time than KnowNo. Mean search time is 0.99 seconds for Ours and 19.99 seconds for KnowNo in WasteSorting and 1.38 and 26.90 seconds in TomatoHarvesting. In TomatoHarvesting, interaction time and total time are 44.99 and 217.82 seconds for KnowNo compared with 34.12 and 180.39 seconds for Ours. Notably, KnowNo uses only 15.43% fewer queries than Ours in TomatoHarvesting with VLM responses compared with 73.22% fewer queries in the oracle experiments. However the similar query counts do not translate into similar execution costs. KnowNo executes longer plans, with 16.10 physical actions compared with 13.27 for Ours. Interaction time and total time for KnowNo are 31.86% and 20.75% longer than for Ours respectively.

To understand the differences from the oracle results, we analyze failed trials and distinguish VLM response errors from baseline planner failures. In WasteSorting, all three failures of Ours involve VLM feedback. For example, the VLM rejects a can classification from perception and the robot places the object in the plastic bin. KnowNo has three VLM response failures (15%) and 17 planner failures (85%). The planner failures occur because KnowNo selects actions without requesting assistance despite unmet action preconditions. In TomatoHarvesting, all three failures of Ours also involve VLM feedback. KnowNo has 18 VLM failures (60%) and 12 planner failures (40%). The



(a) Success rate (%). (b) Query count. (c) Query probability (%). (d) Plan length.

(e) Total time (s). (f) Interaction time (s). (g) Planning time (s).

Fig. 6: Physical robot evaluation with VLM responses. (a)–(d) show success rate, query count, query probability, and plan length for both domains. (e)–(g) show total time, interaction time, and planning time for both domains. Across both domains, our method achieves higher success rates and shorter planning times than KnowNo despite more queries.

Table 3: Physical robot results with VLM and human responses. Parentheses give successful trials over all trials. Other metrics report means over successful trials. A dash marks human results that are not yet available.

Source	Domain	Method	Success (%)	Query count	Query prob. (%)	Plan length	Total time (s)	Interaction time (s)	Planning time (s)
VLM	W.S.	KnowNo	50.00 (20/40)	3.15	28.64	11.00	186.51	15.46	20.08
		Ours	92.50 (37/40)	6.70	19.81	11.24	193.69	26.83	1.11
	T.H.	KnowNo	25.00 (10/40)	7.20	44.10	16.10	217.82	44.99	27.68
		Ours	92.50 (37/40)	8.51	37.63	13.27	180.39	34.12	1.48
Human	W.S.	KnowNo	- (-/10)	-	-	-	-	-	-
		Ours	- (-/10)	-	-	-	-	-	-
	T.H.	KnowNo	- (-/10)	-	-	-	-	-	-
		Ours	- (-/10)	-	-	-	-	-	-

planner failures occur after action selections without external assistance. In six trials, the VLM repeatedly selects detection despite feasible alternatives and the robot reaches the step limit. The remaining 12 VLM failures involve incorrect action selections.

The WasteSorting failures show that a single predicted action can still require additional state verification before execution. In TomatoHarvesting, action selection requires the VLM to reason about robot location and object states to determine the next action. The spatial reasoning burden can lead to less effective action choices. Even successful trials can then require additional queries and longer plans and more total time. Therefore fewer queries do not necessarily reduce interaction time or total time. Ours asks the VLM to verify current state facts and assigns action selection to the planner. The VLM assesses current observations with less need to reason about



Fig. 7: Tablet interface used in the human evaluation. Participants view the robot camera feeds and select responses to robot queries.

future actions and outcomes. Performance may also depend on what the VLM is asked to judge.

Human. We further examine whether the trends in task success and query behavior persist with human feedback. The study is designed for five participants. Recruitment criteria require no difficulties with visual recognition, such as color vision deficiency. The five participants included [number] men and [number] women, with a mean age of [mean] years and an age range of [minimum] to [maximum] years.

We use the same robot platforms and task domains as in the VLM experiments. Participants receive instructions about the task goals and available actions, then answer robot questions according to their own judgment through the tablet interface in Fig. 7. [2]

Each participant completes two trials with each method in each domain, for eight trials per participant and 10 trials per method and domain. Trials are grouped by domain, and the domain and method orders vary across participants to reduce order effects (Table 4). Fig. 8 shows the human results, and Table 3 summarizes both the VLM and human results.

Ours achieves a success rate of 100.00% in WasteSorting and 50.00% in TomatoHarvesting with human responses. KnowNo achieves 16.67% and 33.33% respectively. Mean query counts are 6.00 for Ours and 4.00 for KnowNo in WasteSorting and 9.67 and 4.00 in TomatoHarvesting. Mean planning time is 1.09 seconds for Ours and 20.40 seconds for KnowNo in WasteSorting and

[2]Appendix E provides the detailed participant instructions and trial procedure.

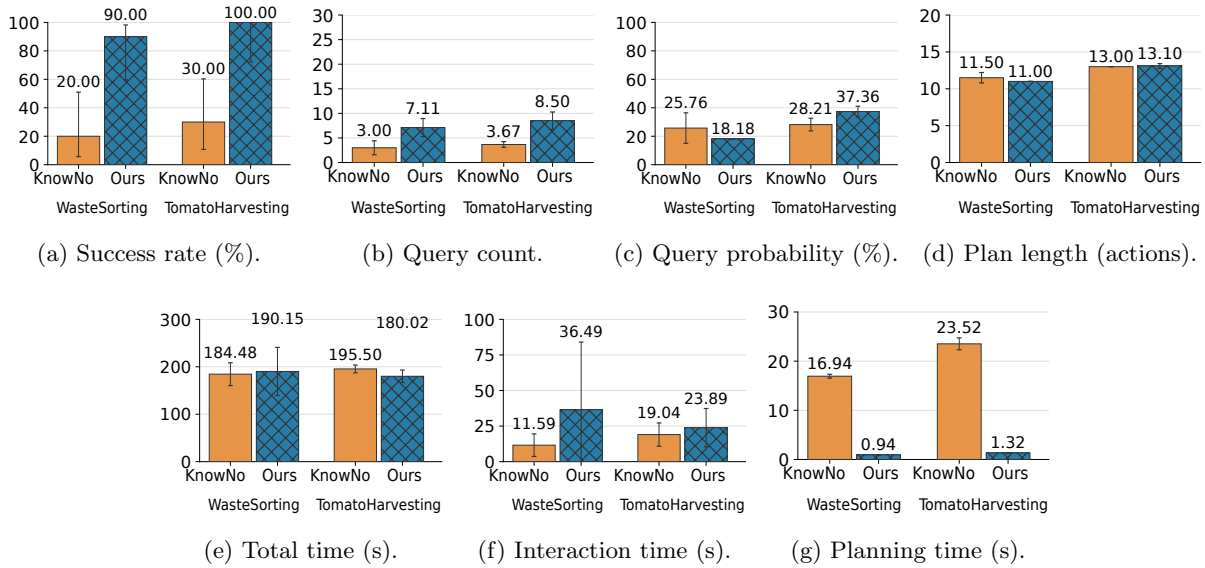
Table 4: Assigned order for the human evaluation. W.S. and T.H. denote WasteSorting and TomatoHarvesting. A denotes Ours and B denotes KnowNo. The method sequence applies to both domains for each participant.

Participant	Assigned order
P01	W.S. $\rightarrow$ T.H. : $A \rightarrow A \rightarrow B \rightarrow B$
P02	T.H. $\rightarrow$ W.S. : $A \rightarrow B \rightarrow A \rightarrow B$
P03	W.S. $\rightarrow$ T.H. : $B \rightarrow B \rightarrow A \rightarrow A$
P04	T.H. $\rightarrow$ W.S. : $B \rightarrow A \rightarrow B \rightarrow A$
P05	W.S. $\rightarrow$ T.H. : $A \rightarrow A \rightarrow B \rightarrow B$

1.46 and 25.40 seconds in TomatoHarvesting. In WasteSorting, interaction time and total time are 15.65 and 167.45 seconds for Ours compared with 10.78 and 183.38 seconds for KnowNo. In TomatoHarvesting, the corresponding times are 27.00 and 182.36 seconds for Ours compared with 16.75 and 181.89 seconds for KnowNo. KnowNo uses 33.33% fewer queries than Ours in WasteSorting and 58.62% fewer in TomatoHarvesting. However the lower query counts do not consistently reduce total time. Plan lengths are similar across methods, with 11.00 physical actions for both methods in WasteSorting and 13.33 for Ours and 13.00 for KnowNo in TomatoHarvesting. Despite the additional queries, Ours requires 8.69% less total time in WasteSorting and a similar total time in TomatoHarvesting.

Ours also achieves higher task success than KnowNo with human responses. These results suggest that query timing determines whether participants have an opportunity to correct robot decisions. Even if a participant notices an incorrect judgment, there may be no opportunity to correct it if the robot proceeds without a query. P01 reported that KnowNo proceeded without a query in situations where they wanted to intervene. Our method requests additional information if belief confidence is insufficient and incorporates human responses into subsequent planning.

Comparison across Feedback Sources . Fig. 9 compares task success and query count across oracle, VLM, and human feedback. Ours achieves higher success rates and uses more queries than KnowNo in both domains across all three feedback sources. However the magnitude of the success advantage depends on the domain and feedback



(a) Success rate (%). (b) Query count. (c) Query probability (%). (d) Plan length (actions).

(e) Total time (s). (f) Interaction time (s). (g) Planning time (s).

Fig. 8: Physical robot evaluation with human responses. (a)–(d) show success rate, query count, query probability, and plan length for WasteSorting and TomatoHarvesting. (e)–(g) show total time, interaction time, and planning time for both domains. An asterisk marks a metric based on a single successful trial, for which the sample standard deviation is undefined.

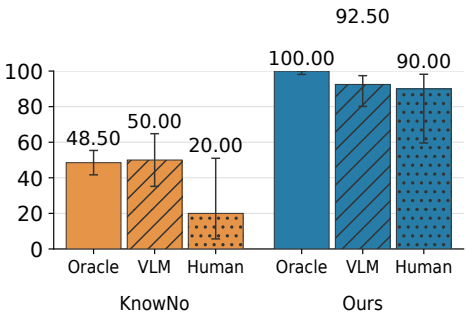
source. In WasteSorting, Ours maintains success rates of 100.00%, 92.50%, and 100.00% with oracle, VLM, and human feedback respectively. In TomatoHarvesting, success decreases from 98.00% with oracle feedback to 92.50% with VLM feedback and 50.00% with human feedback. The success advantage over KnowNo also narrows from 55.00 percentage points with oracle feedback to 16.67 percentage points with human feedback in TomatoHarvesting. Thus the relative advantage of Ours persists, but high success does not persist equally across domains and feedback sources.

Query counts also show differences across feedback sources. In TomatoHarvesting, Ours uses 9.16, 8.51, and 9.67 queries with oracle, VLM, and human feedback respectively. KnowNo uses 2.45, 7.20, and 4.00 queries. The query count of KnowNo increases substantially with VLM feedback, while the query count of Ours remains similar across feedback sources despite the lower success with human feedback. Therefore similar query counts do not imply similar task outcomes, and the number of queries alone does not explain the differences across feedback sources. These differences motivate the subsequent human interview analysis. We examine how participants interpret robot questions and what reasoning participants need to choose a response, with particular attention to question clarity and response effort.

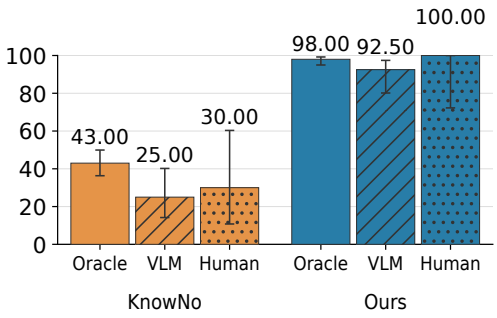
5.2.2 Ablation Study

We conduct an ablation study to assess the contributions of query timing and query selection to task success and query efficiency. We compare seven configurations with oracle feedback in simulation. For concise reference, we use **W** to label query timing variants and **Q** to label query selection variants. With entropy gain selection fixed, we compare random (W1), conformal prediction (W2), and reward-based (W3) timing. With confidence timing fixed, we compare random (Q1) and reward-based (Q2) selection. Fully-Random randomizes both query timing and query selection, while Ours combines confidence timing with entropy gain selection. Table 1 summarizes these configurations. Each configuration has 200 trials per domain. Table 5 and Fig. 10 present the results. Appendix C.3 provides the query policy configurations.

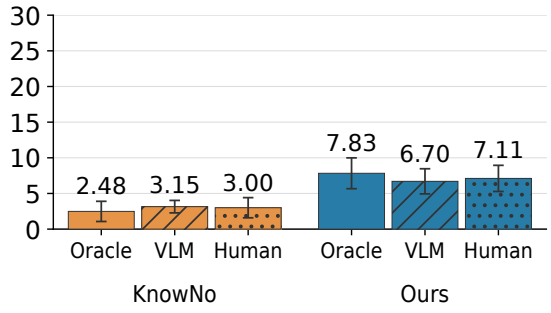
The timing comparison evaluates Fully-Random, W1–W3, and Ours. In WasteSorting,



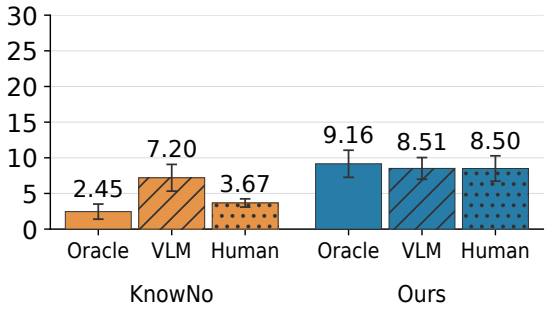
(a) W.S. Success rate (%).



(b) T.H. Success rate (%).



(c) W.S. Query count.



(d) T.H. Query count.

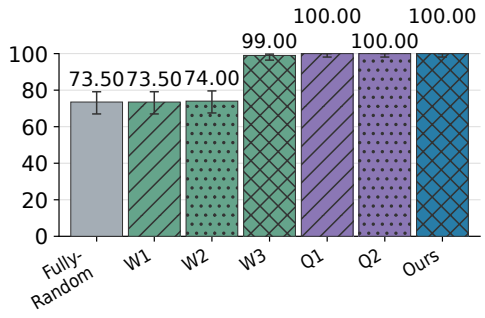
Fig. 9: Comparison across oracle, VLM, and human responses. (a)–(b) show success rates for WasteSorting and TomatoHarvesting, respectively. (c)–(d) show query counts for the same domains in the same order. Oracle trials use simulation, while VLM and human trials use physical robots.

Table 5: Query policy ablation. W1–W3 compare timing criteria and Q1–Q2 compare selection criteria.

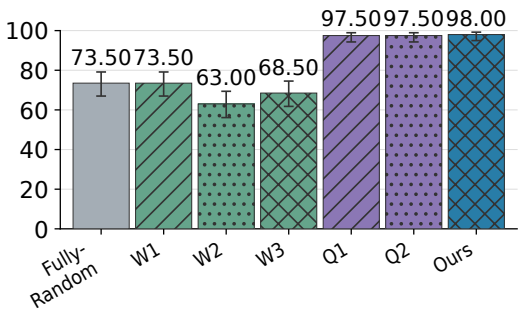
			WasteSorting		TomatoHarvesting	
Condition	Timing	Selection	Success (%)	Queries	Success (%)	Queries
Fully-Random	Random	Random	73.50	10.25 ± 6.11	73.50	9.04 ± 3.94
W1	Random	Entropy Gain	73.50	6.65 ± 3.29	73.50	7.43 ± 2.72
W2	Conformal Prediction	Entropy Gain	74.00	7.09 ± 2.58	63.00	4.05 ± 2.04
W3	Reward	Entropy Gain	99.00	17.65 ± 2.89	68.50	12.16 ± 3.00
Q1	Confidence	Random	100.00	15.21 ± 3.53	97.50	12.68 ± 2.53
Q2	Confidence	Reward	100.00	22.79 ± 6.26	97.50	15.25 ± 2.95
Ours	Confidence	Entropy Gain	100.00	7.83 ± 2.16	98.00	9.16 ± 1.90

Fully-Random and W1 achieve 73.50% success, and W2 achieves 74.00%. W3 achieves 99.00% compared with 100.00% for Ours, but requires 17.65 queries compared with 7.83. Fully-Random, W1, and W2 use 10.25, 6.65, and 7.09 queries respectively. In TomatoHarvesting, Fully-Random and W1 achieve 73.50% success, while W2 and W3 achieve 63.00% and 68.50% compared with 98.00% for Ours. Mean query counts are 9.04 for Fully-Random, 7.43 for W1, 4.05 for W2, 12.16 for W3, and 9.16 for Ours.

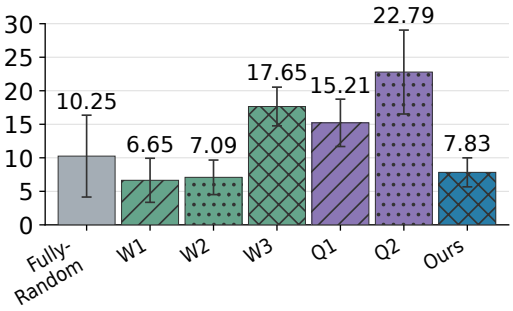
These results show that task success depends on the query trigger as well as query selection. W1 replaces random selection in Fully-Random with entropy gain and reduces query count without improving success. Random timing can miss



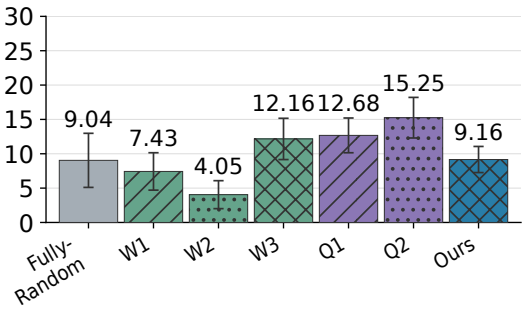
(a) W.S. Success rate (%).



(b) T.H. Success rate (%).



(c) W.S. Query count.



(d) T.H. Query count.

Fig. 10: Query timing and selection ablation. (a)–(b) show success rates for WasteSorting and TomatoHarvesting, respectively. (c)–(d) show query counts for the same domains in the same order. W1–W3 vary query timing, while Q1–Q2 vary query selection. The results suggest that timing based on belief confidence supports task success, while selection by expected entropy reduction reduces the number of queries required.

necessary verification because the trigger does not depend on current state uncertainty. W2 can also end queries after the action prediction set narrows to one candidate, even if the state facts required to execute the candidate action remain uncertain. W3 uses expected task reward to determine whether to initiate queries, then selects queries by entropy gain and repeats verification until confidence reaches τ. In WasteSorting, object category verification directly informs the correct bin choice and task reward, so repeated verification may support the high success rate of 99.00%. However W3 requires 125.43% more queries than Ours. In TomatoHarvesting, W3 uses 32.71% more queries than Ours, but the success rate of W3 is 29.50 percentage points lower. Therefore reward-based timing can request more queries overall and still fail to verify necessary state information at the appropriate time. Thus Ours uses current state confidence as the query trigger and achieves the highest success in both domains with fewer queries than reward-based timing.

With confidence-based query timing fixed, different query selection strategies maintain high success rates but require different numbers of queries. The query selection comparison evaluates Fully-Random, Q1, Q2, and Ours. Fully-Random achieves 73.50% success in both domains with 10.25 queries in WasteSorting and 9.04 in TomatoHarvesting. Q1 retains random selection but uses confidence timing, and success increases to 100.00% and 97.50% respectively. With confidence timing fixed, Q1, Q2, and Ours all achieve 100.00% success in WasteSorting. Mean query counts are 15.21 for Q1, 22.79 for Q2, and 7.83 for Ours. In TomatoHarvesting, Q1 and Q2 achieve 97.50% success compared with 98.00% for Ours. Mean

query counts are 12.68 for Q1, 15.25 for Q2, and 9.16 for Ours.

Confidence timing maintains 100.00% success in WasteSorting and at least 97.50% in TomatoHarvesting across all three query selection strategies. Under confidence timing, query selection primarily affects the number of queries needed to reach the required confidence. Random selection does not prioritize facts by their expected reduction in state uncertainty. Reward-based selection evaluates expected task reward without directly measuring the information provided by a query response. Entropy gain selects facts according to the expected reduction in state uncertainty, so the selection criterion matches the confidence requirement that ends a query episode. Relative to Q1 and Q2, Ours reduces query count by 48.54% and 65.64% in WasteSorting and by 27.72% and 39.92% in TomatoHarvesting. Thus our query selection method reduces the number of queries required under confidence timing without reducing task success.

These results support the main contribution of our method through the distinct roles of query timing and query selection. Query timing determines whether the robot requests additional information before it executes actions under state uncertainty. Confidence timing achieves high success with random, reward-based, and entropy gain selection, while entropy gain under random timing reduces query count without improving success over Fully-Random. Once confidence timing establishes a high success level, entropy gain reduces the number of queries needed to reach the required confidence. Therefore our method combines a timing criterion that supports task success with a selection criterion that reduces query count at that success level.

5.2.3 Threshold Experiments

After the ablation comparison, we examine how the confidence threshold controls task success and query count. We vary τ from 0.0 to 1.0 in increments of 0.1 with entropy gain selection fixed and correct oracle responses in simulation. Each threshold has 200 trials per domain. Table 6 and Fig. 11 present the results. The results at $\tau = 0.8$ use the same Ours trials as the baseline and ablation comparisons, as specified in Appendix C.4.

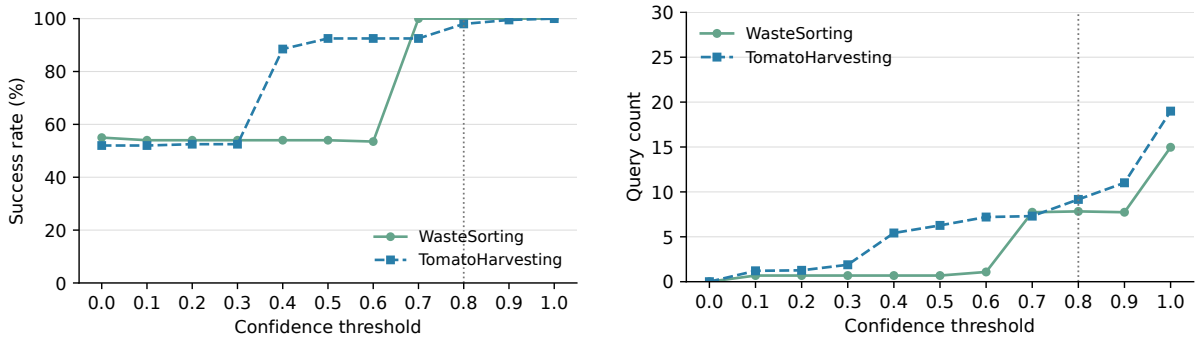
Table 6: Task success and query count across confidence thresholds.

	WasteSorting		TomatoHarvesting	
τ	Success (%)	Queries	Success (%)	Queries
0.0	55.00	0.00 ± 0.00	52.00	0.00 ± 0.00
0.1	54.00	0.69 ± 0.84	52.00	1.21 ± 1.43
0.2	54.00	0.69 ± 0.84	52.50	1.27 ± 1.42
0.3	54.00	0.69 ± 0.84	52.50	1.89 ± 1.83
0.4	54.00	0.69 ± 0.84	88.50	5.42 ± 2.30
0.5	54.00	0.69 ± 0.84	92.50	6.27 ± 2.07
0.6	53.50	1.08 ± 0.97	92.50	7.20 ± 1.99
0.7	100.00	7.74 ± 2.25	92.50	7.30 ± 2.04
0.8	100.00	7.83 ± 2.16	98.00	9.16 ± 1.90
0.9	100.00	7.74 ± 2.25	99.50	11.02 ± 1.52
1.0	100.00	14.97 ± 2.02	100.00	19.00 ± 1.43

In WasteSorting, success improves sharply at $\tau = 0.7$ and remains constant at higher thresholds. With $\tau = 0.0$, the robot makes no queries and achieves 55.00% success. Thresholds from 0.1 to 0.6 yield success rates between 53.50% and 54.00% with at most 1.08 queries on average. At $\tau = 0.7$, success reaches 100.00% with 7.74 queries. Success remains 100.00% over $\tau = 0.7$–0.9, and query count stays between 7.74 and 7.83. However $\tau = 1.0$ requires 14.97 queries without further success improvement. The additional queries confirm that the strictest confidence requirement can increase interaction frequency after task success reaches its maximum.

In TomatoHarvesting, higher thresholds improve success over a wider range. Success is 52.00% without queries at $\tau = 0.0$ and remains at most 52.50% through $\tau = 0.3$. At $\tau = 0.4$, success increases to 88.50% with 5.42 queries. Success reaches 92.50% over $\tau = 0.5$–0.7 and 98.00% at $\tau = 0.8$ with 9.16 queries. Raising τ to 0.9 increases success to 99.50% with 11.02 queries. At $\tau = 1.0$, success reaches 100.00% with 19.00 queries. Therefore the increase from 0.9 to 1.0 adds 7.98 queries for only 0.50 percentage points of additional success.

The two domains require different confidence levels to reach high success, but both show limited gains from the strictest threshold. Relative



(a) Task success.

(b) Mean query count.

Fig. 11: Effect of the confidence threshold on task success and query count. (a) shows success rates and (b) shows mean query counts for WasteSorting and TomatoHarvesting. The dotted line marks the default threshold of $\tau = 0.8$. Compared with $\tau = 0.8$, $\tau = 1.0$ nearly doubles query count with little or no improvement in task success.

to $\tau = 0.8$, $\tau = 1.0$ increases query count by 91.19% in WasteSorting and 107.30% in TomatoHarvesting. Success improves by 0.00 and 2.00 percentage points respectively. We use $\tau = 0.8$ as a common threshold because it achieves 100.00% and 98.00% success in the two domains with substantially fewer queries than $\tau = 1.0$. Thus the threshold controls the amount of state verification, and the highest confidence requirement does not provide a proportional improvement in task success.

5.2.4 Interviews

We interview participants to examine how participants understood and answered queries after they experienced both methods in WasteSorting and TomatoHarvesting.

Participants described different judgments required by the two methods. P01 explained that Ours required only an assessment of the current situation, without the need to anticipate the next action. For KnowNo, P01 considered which actions had to occur first and whether a proposed action could be executed at that point. P01 also found it unclear whether an action choice implied that the robot would perform the necessary preceding actions. P02 described the questions from Ours as simpler and reported less need to think before answering.

Perceived effort across the entire task differed between participants. P01 found Ours more laborious because questions occurred more frequently, but reported greater satisfaction because the judgments were clearer and execution followed the intended course. P02 described Ours as less laborious and felt that the robot made more of the decisions. Thus, easier individual judgments did not imply uniformly lower effort across the task.

Participants also described difficulties with the information provided for their judgments. P01 reported that a mismatch between object identifiers in the question and the image made the referenced tomato unclear. P02 reported that occlusion by the gripper made it difficult to judge the condition of a tomato from the camera view. P01 also requested a way to revise an earlier answer before the robot executed an action.

5.2.5 Discussion

Results with oracle, VLM, and human responses support the effectiveness of our method. Query timing and query selection serve complementary roles. The confidence threshold determines when to query based on state uncertainty, while expected entropy reduction determines which information to request. The ablation experiments highlight the importance of both decisions. Effective query selection can reduce the number of queries, but reliable execution also depends on requesting information when state is uncertain.

The objective is to obtain sufficient state information for successful execution while limiting the number of queries. A stricter confidence requirement can increase the number of queries without a corresponding improvement in task success. Thus, reducing uncertainty beyond what is needed for successful execution may introduce unnecessary interaction. These findings suggest that effective querying requires balancing the information needed for reliable execution against the cost of obtaining it.

Beyond task performance, the interviews also provide indirect insight into *how* to query people. Participants described our state questions as easier to judge and reported less need to think before answering than with KnowNo. This feedback supports the use of the proposed state questions for human assistance. However, *how* also includes interaction modalities and UI design, which can affect usability and cognitive workload. We did not compare voice interfaces or alternative UI designs, and their effects on human interaction remain future work.

6 Conclusion and Future Work

We presented a method that uses state uncertainty to determine when and what to query during robot task execution. Belief confidence determines query timing, and expected entropy reduction guides the selection of state facts. Experiments in WasteSorting and TomatoHarvesting show that query timing supports task success, while query selection reduces the number of required queries. Future work will examine multi-modal interfaces and UI design for human interaction.

Acknowledgements This research was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(No. RS-2024-00461583) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00411007).

Ethics approval and consent This study was approved by the Institutional Review Board of Sogang University (Approval No. SGUIRB-A-2501-04-2). The study was conducted in accordance with the ethical standards of the institution. Informed consent was obtained from all participants before participation.

References

Anand, Y., et al.: Adaptive querying for reward learning from human feedback. *Frontiers in Robotics and AI* **12**, 1734564 (2025)

Burks, L., et al.: Collaborative human-autonomy semantic sensing through structured POMDP planning. *Robotics and Autonomous Systems* **140**, 103753 (2021)

Chen, J.Y.C., Barnes, M.J.: Human–agent teaming for multirobot control: A review of human factors issues. *IEEE Transactions on Human-Machine Systems* **44**(1), 13–29 (2014)

Carion, N., et al.: SAM 3: Segment anything with concepts. arXiv preprint arXiv:2511.16719 (2025). https://arxiv.org/abs/2511.16719

Chitta, S., et al.: MoveIt! [ROS topics]. *IEEE Robotics & Automation Magazine* **19**(1), 18–19 (2012)

Dragan, A.D., et al.: Legibility and predictability of robot motion. In: *International Conference on Human-Robot Interaction (HRI)*, pp. 301–308 (2013). IEEE

Davis, R., et al.: What is a knowledge representation? *AI Magazine* **14**(1), 17–17 (1993)

Feng, Z., Hansen, E.: Approximate planning for factored POMDPs. In: *Proceedings of the 6th European Conference on Planning* (2001)

Fikes, R.E., Nilsson, N.J.: STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence* **2**(3–4), 189–208 (1971)

Gemma Team: Gemma 4 technical report. arXiv preprint arXiv:2607.02770 (2026). https://doi.org/10.48550/arXiv.2607.02770

Ghallab, M., et al.: PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control (1998). https://www.cs.cmu.edu/%7Emmv/planning/readings/98aips-PDDL.pdf

Goodrich, M.A., Schultz, A.C.: Human–robot

interaction: a survey. *Foundations and Trends in Human–Computer Interaction* **1**(3), 203–275 (2008)

Helmert, M.: Concise finite-domain representations for PDDL planning tasks. *Artificial Intelligence* **173**(5–6), 503–535 (2009)

Hansen, E.A., Feng, Z.: Dynamic programming for POMDPs using a factored state representation. In: *Proceedings of the International Conference on Artificial Intelligence Planning Systems (AIPS)*, pp. 130–139 (2000)

Igbinedion, I., Karaman, S.: Learning when to ask for help: Efficient interactive navigation via implicit uncertainty estimation. arXiv preprint arXiv:2305.16502 (2024). https://arxiv.org/abs/2305.16502

Kaipa, K.N., et al.: Addressing perception uncertainty induced failure modes in robotic bin-picking. *Robotics and Computer-Integrated Manufacturing* **42**, 17–38 (2016)

Kaelbling, L.P., et al.: Planning and acting in partially observable stochastic domains. *Artificial Intelligence* **101**(1–2), 99–134 (1998)

Kim, S.-K., et al.: PLGRIM: Hierarchical value learning for large-scale exploration in unknown environments. In: *Proceedings of the 31st International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, pp. 652–662 (2021)

Kurniawati, H., Yadav, V.: An online POMDP solver for uncertainty planning in dynamic environment. In: *Robotics Research: The 16th International Symposium ISRR*, pp. 611–629 (2016). Springer

Lauri, M., Ritala, R.: Planning for robotic exploration based on forward simulation. *Robotics and Autonomous Systems* **83**, 15–31 (2016)

Liang, K., et al.: Introspective planning: Aligning robots' uncertainty with inherent task ambiguity. *Advances in Neural Information Processing Systems* **37**, 71998–72031 (2024)

Li, M., et al.: Uncertainty-aware neural-symbolic task planning with balanced exploration and execution for domestic service robot. Expert Systems with Applications, 132962 (2026)

Mullen, J.F., Manocha, D.: LBAP: Improved uncertainty alignment of LLM planners using bayesian inference. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 18716–18723 (2025). IEEE

Mangalindan, D.H., et al.: When to seek help: Trust-aware assistance-seeking in human-supervised autonomy. *ACM Transactions on Human-Robot Interaction* **15**(2), 1–23 (2025)

Murphy, K., Weiss, Y.: The factored frontier algorithm for approximate inference in DBNs. arXiv preprint arXiv:1301.2296 (2013). https://arxiv.org/abs/1301.2296

Pineau, J., et al.: Point-based value iteration: An anytime algorithm for POMDPs. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 3, pp. 1025–1032 (2003)

Pajarinen, J., Kyrki, V.: Robotic manipulation of multiple objects as a POMDP. *Artificial Intelligence* **247**, 213–228 (2017)

Pajarinen, J., et al.: POMDP planning under object composition uncertainty: Application to robotic manipulation. *IEEE Transactions on Robotics* **39**(1), 41–56 (2023) https://doi.org/10.1109/TRO.2022.3188168

Qu, K., et al.: Stochastic cumulative DNN inference with RL-aided adaptive IoT device-edge collaboration. *IEEE Internet of Things Journal* **10**(20), 18000–18015 (2023) https://doi.org/10.1109/JIOT.2023.3280746

Ren, A.Z., et al.: Robots that ask for help: Uncertainty alignment for large language model planners. In: *Proceedings of the Conference on Robot Learning (CoRL)* (2023)

Ross, S., et al.: Online planning algorithms for POMDPs. *Journal of Artificial Intelligence Research* **32**, 663–704 (2008)

Singi, S., et al.: Decision making for human-in-the-loop robotic agents via uncertainty-aware reinforcement learning. arXiv preprint arXiv:2303.06710 (2023). https://arxiv.org/abs/2303.06710

Sanner, S., Kersting, K.: Symbolic dynamic programming for first-order POMDPs. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, pp. 1140–1146 (2010)

Serrano, S.A., et al.: Knowledge-based hierarchical POMDPs for task planning. *Journal of Intelligent & Robotic Systems* **101**(4), 82 (2021)

Silver, D., Veness, J.: Monte-carlo planning in large POMDPs. Advances in Neural Information Processing Systems **23** (2010)

Somani, A., et al.: DESPOT: Online POMDP planning with regularization. Advances in Neural Information Processing Systems **26** (2013)

Tian, Y., et al.: YOLOv12: Attention-centric real-time object detectors. arXiv preprint arXiv:2502.12524 (2025). https://arxiv.org/abs/2502.12524

Veiga, T., et al.: Point-based POMDP solving with factored value function approximation. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28 (2014)

Xiao, Y., et al.: Online planning for target object search in clutter under partial observability. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8241–8247 (2019). IEEE

Zhang, S., Stone, P.: CORPP: Commonsense reasoning and probabilistic planning, as applied to dialog with a mobile robot. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29 (2015)

Zhao, M., et al.: Conformalized interactive imitation learning: Handling expert shift and intermittent feedback. In: *International Conference on Learning Representations (ICLR)* (2025)

Appendix Contents

The appendix provides planning algorithms, reproducibility settings, evaluation details, and the human interaction procedure.

1. Appendix A. Planning algorithms
   - Appendix A.1. POMCP search
   - Appendix A.2. Simulation with applicable actions
   - Appendix A.3. Action selection and rollout
2. Appendix B. Baseline algorithms
   - Appendix B.1. KnowNo, IntroPlan, and Query-Action
3. Appendix C. Detailed experimental settings
   - Appendix C.1. Planner and evaluation parameters
   - Appendix C.2. Baseline configurations
   - Appendix C.3. Ablation configurations
   - Appendix C.4. Shared oracle reference
   - Appendix C.5. Domain model parameters
   - Appendix C.6. Randomization and seeds
   - Appendix C.7. Metric and time definitions
   - Appendix C.8. External VLM requests
4. Appendix D. Supplementary evaluation details
   - Appendix D.1. Threshold protocol
   - Appendix D.2. Ablation reporting
   - Appendix D.3. VLM failure classification
   - Appendix D.4. KnowNo action requests
5. Appendix E. Human evaluation procedure

Appendix A Planning Algorithms

This appendix provides the POMCP search procedures used in Sec. 4.1. The search budget is n_{sim}, the maximum simulation depth is $d_{\max}$, the discount cutoff is ϵ, and the UCB exploration constant is c. The search tree T stores visit counts N, value estimates V, and sampled state particles $B(h)$ at history h.

A.1 POMCP Search with Applicable Actions

Algorithm 2 describes POMCP search with a fixed simulation budget. Each simulation samples a state from the particles stored at the root history. If the root contains no particles, the simulation starts from the committed knowledge state. These search-tree particles support action-value estimation and are separate from the weighted frontier used for confidence and query selection in Sec. 4.1.1. After the simulations, the planner selects the highest-value action applicable to the committed state.

Algorithm 2: POMCP Search with Applicable Actions

Input belief b, simulation budget n_{sim}
Output selected action $a \in \mathcal{A}$

procedure SEARCH(b)
  $h \leftarrow$ root history
  $s^K \leftarrow \mathit{knowledge}(b)$
  for $i \leftarrow 1$ **to** n_{sim} **do**
    if $B(h) \neq \emptyset$ **then**
      sample $s \sim B(h)$
    else
      $s \leftarrow s^K$
    SIMULATE($s, h, 0$)
  $a \leftarrow$ SEARCH-BEST(h, s^K, false)
  return a

A.2 POMCP Simulation with Applicable Actions

Algorithm 3 describes how simulations are constrained by symbolic action applicability. If a history node is expanded for the first time, only actions whose preconditions hold in the sampled symbolic state are inserted into the search tree. The generative model then samples the next state, observation, and reward while preserving the standard POMCP value-update structure.

Algorithm 3: POMCP Simulation with Applicable Actions

Input state s, history h, depth $depth$, maximum depth $d_{\max}$
Output simulated return R

procedure SIMULATE$(s, h, depth)$
  if $depth > 0$ *and* $(depth \geq d_{\max}$ *or* $\gamma = 0$ *or* $\gamma^{depth} < \epsilon)$ **then**
    return 0
  if *there is no* $a[\![s]\!] \in \mathcal{A}$ **then**
    $N(h) \leftarrow N(h) + 1$
    return 0
  if *history* h *has no action children* **then**
    foreach $a[\![s]\!] \in \mathcal{A}$ **do**
      $T, N(ha), V(ha) \leftarrow T \cup \{ha\}, 0, 0$
    $N(h) \leftarrow N(h) + 1$
    $V(h) \leftarrow$ROLLOUT$(s, depth)$
    return $V(h)$
  $a \leftarrow$ SEARCH-BEST(h, s, true)
  sample (s', o, r) from simulator $\mathcal{G}(s, a)$
  $R \leftarrow r + \gamma \cdot$ SIMULATE$(s', hao, depth + 1)$
  $B(h) \leftarrow B(h) \cup \{s\}$
  $N(h) \leftarrow N(h) + 1$
  $N(ha) \leftarrow N(ha) + 1$
  $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$
  return R

A.3 Action Selection and Rollout

Algorithm 4 specifies the shared action-selection routine used during tree search and rollout. During simulation, unvisited applicable actions receive priority and visited actions are compared using UCB. The final root decision uses the current value estimate. Rollout actions are sampled uniformly from applicable actions. This keeps both planning and rollout consistent with the symbolic transition model.

Algorithm 4: Action Selection and Rollout

Input history h, state s

procedure SEARCH-BEST(h, s, use_ucb)

  $\mathcal{C} \leftarrow \{a \mid a[\![s]\!] \in \mathcal{A},\ ha \in T\}$

  if $\mathcal{C} = \emptyset$ **then**

    return $\emptyset$

  if *use_ucb* **then**

    if *an action* $a \in \mathcal{C}$ *has* $N(ha) = 0$ **then**

      return an unvisited action in $\mathcal{C}$

    return $\arg\max_{a \in \mathcal{C}} \left[V(ha) + c\sqrt{\frac{\log(\max\{1, N(h)\})}{N(ha)}} \right]$

  else

    return $\arg\max_{a \in \mathcal{C}} V(ha)$

procedure ROLLOUT($s, depth$)

  if $depth > 0$ *and* $(depth \geq d_{\max}$ *or* $\gamma = 0$ *or* $\gamma^{depth} < \epsilon)$ **then**

    return 0

  if *there is no* $a[\![s]\!] \in \mathcal{A}$ **then**

    return 0

  $a \sim \text{Uniform}(\{a \mid a[\![s]\!] \in \mathcal{A}\})$

  sample s' from $T(\cdot \mid s, a)$ and evaluate reward r

  return $r + \gamma \cdot$ ROLLOUT($s', depth + 1$)

Appendix B Baseline Algorithms

This appendix provides separate execution procedures for KnowNo, IntroPlan, and Query-Action in Table 1. Appendix C.2 gives the experimental parameter values.

We use $\mathcal{D}$ for generated action candidates, $\sigma(a)$ for temperature-scaled candidate scores, and $\mathcal{P}$ for the conformal prediction set. The symbol $\perp$ denotes the option that requests an action outside the listed candidates. An external action-choice request returns one action, including a manually supplied action if the fallback option is selected. The query function in Algorithm 1 returns the Boolean truth value of a state fact. The function UpdateKnowledge records observed action outcomes without constructing the weighted belief used by Ours.

B.1 Baseline Procedures

B.1.1 KnowNo

Algorithm 5 requests one action choice if the conformal set does not identify a single concrete action. A singleton action is executed without an external query.

Algorithm 5: KnowNo

Input initial knowledge s_0^K, task goal G, calibration threshold $\hat{q}$, step limit $T_{\max}$
Output executed physical action sequence π

procedure KNOWNO($s_0^K, G, \hat{q}, T_{\max}$)
  $s^K \leftarrow s_0^K \quad t \leftarrow 0 \quad \pi \leftarrow ()$
  while $t < T_{\max}$ *and the trial is not terminal* **do**
    $\mathcal{D} \leftarrow \text{GenerateActions}(s^K, G, \pi)$
    $\sigma \leftarrow \text{ScoreActions}(s^K, G, \pi, \mathcal{D})$
    $\mathcal{P} \leftarrow \{a \in \mathcal{D} \mid \sigma(a) \geq 1 - \hat{q}\}$
    if $|\mathcal{P}| = 1$ *and* $\perp \notin \mathcal{P}$ **then**
      $a \leftarrow$ the action in $\mathcal{P}$
    else
      $a \leftarrow \text{QueryActionChoice}(\mathcal{D}, \mathcal{P})$
    if $a = \emptyset$ **then**
      break
    execute a and receive observation o
    $s^K \leftarrow \text{UpdateKnowledge}(s^K, a, o)$
    $\pi \leftarrow \pi \parallel a \quad t \leftarrow t + 1$
  return π

B.1.2 IntroPlan

Algorithm 6 extends the candidate evaluation of KnowNo with retrieved examples and an introspective explanation. Retrieval uses lexical cosine similarity over example contexts. The explanation informs candidate scoring before the conformal decision.

Algorithm 6: IntroPlan

Input initial knowledge s_0^K, goal G, example library $\mathcal{E}$, calibration threshold $\hat{q}$, step limit $T_{\max}$
Output executed physical action sequence π

procedure INTROPLAN($s_0^K, G, \mathcal{E}, \hat{q}, T_{\max}$)
  $s^K \leftarrow s_0^K$ $t \leftarrow 0$ $\pi \leftarrow ()$
  while $t < T_{\max}$ *and the trial is not terminal* **do**
    $\mathcal{D} \leftarrow$ GenerateActions(s^K, G, π)
    $\mathcal{E}_3 \leftarrow$ RetrieveExamples($s^K, G, \pi, \mathcal{D}, \mathcal{E}, 3$)
    $e \leftarrow$ ExplainCandidates($s^K, G, \pi, \mathcal{D}, \mathcal{E}_3$)
    remove any direct prediction from explanation e
    $\sigma \leftarrow$ ScoreActions($s^K, G, \pi, \mathcal{D}, e$)
    $\mathcal{P} \leftarrow \{a \in \mathcal{D} \mid \sigma(a) \geq 1 - \hat{q}\}$
    if $|\mathcal{P}| = 1$ *and* $\perp \notin \mathcal{P}$ **then**
      $a \leftarrow$ the action in $\mathcal{P}$
    else
      $a \leftarrow$ QueryActionChoice($\mathcal{D}, \mathcal{P}$)
    if $a = \emptyset$ **then**
      break
    execute a and receive observation o
    $s^K \leftarrow$ UpdateKnowledge(s^K, a, o)
    $\pi \leftarrow \pi \,\|\, a$ $t \leftarrow t + 1$
  return π

B.1.3 Query-Action

Algorithm 7 places Boolean queries in the POMCP action space. Queries and physical actions share the total decision limit. The posterior remains available after a query or physical observation, and the MAP state supplies the symbolic knowledge used to evaluate root applicability. Each search rebuilds the root from the current posterior and evaluates queries and physical actions by expected cumulative reward. Simulated query responses update the belief without changing the physical state.

Algorithm 7: Query-Action

Input initial knowledge s_0^K, initial belief $\mathcal{B}_0$, total decision limit $T_{\max}$
Output executed physical action sequence π

procedure QUERY-ACTION($s_0^K, \mathcal{B}_0, T_{\max}$)
  $s^K \leftarrow s_0^K$ $\mathcal{B} \leftarrow \mathcal{B}_0$ $d \leftarrow 0$ $\pi \leftarrow ()$
  while $d < T_{\max}$ *and the trial is not terminal* **do**
    $\mathcal{A}_q \leftarrow$ applicable Boolean queries about ambiguous facts in $\mathcal{B}$
    $\mathcal{A}_{\text{root}} \leftarrow \mathcal{A}(s^K) \cup \mathcal{A}_q$
    $a \leftarrow$ SearchQueryPOMCP($\mathcal{B}, \mathcal{A}_{\text{root}}$)
    if $a = \emptyset$ **then**
      break
    $d \leftarrow d + 1$
    if a *is a query about fact* c **then**
      $y \leftarrow$ `QUERY`(c)
      $\mathcal{B} \leftarrow$ `FILTER`$(\mathcal{B}, c, y)$
    else
      execute a and receive observation o
      $\mathcal{B} \leftarrow$ `UPDATE-BELIEF`(s^K, a, o)
      $\pi \leftarrow \pi \parallel a$
    $s^K \leftarrow \arg\max_{s^k \in \mathcal{B}} w_k$
  return π

Appendix C Detailed Experimental Settings

This appendix specifies the planner settings, query policies, domain models, and reporting conventions for Sec. 5.1. The oracle experiments use simulated execution with correct responses. The VLM and human evaluations use physical robot execution. We aggregate trials within each domain for all reported comparisons.

C.1 Planner and Evaluation Parameters

Table C1 lists the default POMCP parameters for Ours. The oracle trial limit is 50 physical actions, and the physical trial limit is 20. Queries do not consume physical steps for Ours. Query-Action uses a limit of 50 total decisions that includes queries and physical actions. The reward-based configurations use the discount factors specified below.

C.2 Baseline Configurations

The oracle baseline comparison evaluates KnowNo, IntroPlan, and Query-Action against Ours. The physical experiments compare KnowNo and Ours. Appendix B.1 provides separate procedures for the three baselines. KnowNo and IntroPlan use GPT-4o for action generation and candidate scoring. This planning model is separate from Gemma 4 31B IT, which provides external feedback in the VLM experiments.

The action generation temperature is 0.3, and the candidate score temperature is 5.0. KnowNo uses calibration thresholds $\hat{q} = 0.845165$ for WasteSorting and $\hat{q} = 0.901119$ for TomatoHarvesting in the

Table C1: Planner and evaluation parameters used by Ours.

Parameter	**Value**
POMCP simulations per planning step	100
Maximum simulation depth	20
Discount factor γ	0.2
UCB exploration constant c	1.0
Oracle physical action limit	50
Physical action limit	20
Default confidence threshold τ	0.8
Discount cutoff ϵ	0.005
Oracle frontier capacity	8,000
Oracle search-node particle capacity	8,000
Oracle transition sampling budget	250
Oracle initial configurations per domain	5
Trials per initial configuration and condition	40
Oracle trials per domain and condition	200

oracle comparison. The physical VLM logs contain the same values except for one WasteSorting trial with $\hat{q} = 0.8704$. IntroPlan retrieves three examples and uses calibration thresholds 0.980493 and 0.985552 for the two domains in the same order. Query-Action uses the configuration selected by a separate parameter search. The discount factor is 0.9 for WasteSorting and 0.5 for TomatoHarvesting. Query cost is zero, the failure penalty is 10, assumed response accuracy is 1.0, and the planner uses 100 simulations per decision.

C.3 Ablation Configurations

Fully-Random uses random timing and random selection. W1, W2, and W3 use entropy gain selection with random, conformal prediction, and reward-based timing respectively. Q1 and Q2 use confidence timing with random and reward-based selection respectively. Random timing initiates a query episode with probability 0.4 per physical step. Random selection samples uniformly from facts that distinguish the current state hypotheses. Confidence timing initiates verification if belief confidence falls below $\tau = 0.8$. After a trigger, Fully-Random, W1, Q1, Q2, and Ours continue verification until confidence reaches τ or no differentiating fact remains.

W2 initiates verification if the conformal action prediction set contains multiple candidates. The criterion is reevaluated after each response. The calibration thresholds are 0.8704 for WasteSorting and 0.8404 for TomatoHarvesting, with score temperature 5.0. Four unparseable outputs count as failures in the success rate and contribute no successful-trial cost measurements.

W3 evaluates the expected cumulative reward of query and physical actions. A query episode begins if the highest query value exceeds the highest physical action value. After the initial trigger, entropy gain selects questions and confidence determines whether verification continues. Q2 starts queries using confidence and selects the fact with the highest query value. Q2 does not compare that value with physical action values to cancel a query. Both reward-based variants use zero query cost, a failure penalty of 10, and assumed response accuracy 1.0. The discount factors are 0.9 for WasteSorting and 0.5 for TomatoHarvesting and apply to query evaluation and physical action planning. The query evaluator uses at least 100 simulations and increases the budget to the number of root candidates plus one if necessary. The remaining configurations use $\gamma = 0.2$.

C.4 Oracle Reference Trials

We use the complete Ours batch from the policy ablation as a shared oracle reference for the baseline and ablation comparisons as well as the threshold analysis at $\tau = 0.8$. This reference contains 200 trials per domain and is reused across these comparisons. The other threshold points retain their original trial batches.

C.5 Domain Model Probabilities

Table C2 lists the parameters in the domain configuration files. Transition probabilities determine symbolic successor hypotheses. Observation sampling generates simulated sensor responses, and observation likelihoods weight the represented hypotheses. Detection sampling accuracy and likelihood accuracy are specified separately. Physical execution supplies real sensor observations to the belief update.

Table C2: Domain model parameters for transition and observation models.

Domain	Model component	Probability
WasteSorting	Detect observed transition	0.90
WasteSorting	Detect category transition	0.50
WasteSorting	Pick transition	0.90
WasteSorting	Place transition	0.90
WasteSorting	Detect observation likelihood	0.97
WasteSorting	Detect category observation	0.85
WasteSorting	Pick/place observation	0.95
WasteSorting	Detect sampling accuracy	$[0.94, 1.00]$
WasteSorting	False-positive likelihood	0.03
TomatoHarvesting	Navigate transition	0.99
TomatoHarvesting	Detect transition	0.95
TomatoHarvesting	Pick transition	0.95
TomatoHarvesting	Scan transition	0.90
TomatoHarvesting	Place/discard transition	0.99
TomatoHarvesting	Detect observation likelihood	0.90
TomatoHarvesting	Detect classification observation	0.95
TomatoHarvesting	Scan observation	0.85
TomatoHarvesting	Detect sampling accuracy	$[0.85, 0.95]$
TomatoHarvesting	False-positive probability	0.03
TomatoHarvesting	Navigate observation	0.95
TomatoHarvesting	Place observation	1.00

The WasteSorting category transition parameter 0.50 creates competing category hypotheses after detection. Detection observation sampling uses the hidden object category, with uniform allocation of classification error across the other categories. Occluded objects and objects already held or placed in bins are excluded from simulated detection. In TomatoHarvesting, detection produces ripe or unripe labels, and rotten tomatoes share the ripe visual label. A subsequent scan distinguishes fresh from rotten tomatoes. The detect models combine individual object outcomes into a joint observation distribution. Detector confidence values do not directly scale the symbolic observation likelihood.

C.6 Randomization and Episode Seeds

Each oracle trial records a pseudorandom seed. The runner uses the seed for Python `random`, NumPy sampling, initial arrangements, and randomized query decisions. Conditions within each oracle experiment share the seed assigned to the same domain, initial configuration, and repetition. Different policies can consume random numbers differently after different action choices. The shared Ours reference described in Appendix C.4 retains its original seeds.

C.7 Uncertainty Estimates and Time Measures

Success rates use all attempted trials. Query counts, query probabilities, plan lengths, and time measures use successful trials and are reported as mean ± sample standard deviation. Query probability is the fraction of physical steps that initiate at least one query. Several questions within one query episode increase query count but contribute only one triggered step. Success bars show 95% Wilson confidence intervals. Cost error bars show sample standard deviations. A dash or the asterisk identified in the figure caption denotes a single successful trial for which sample standard deviation is unavailable.

Search time is the sum of directly recorded planning durations within a trial. The oracle plan-time comparison uses cumulative POMCP search time for Ours and Query-Action and cumulative candidate generation and scoring time for KnowNo and IntroPlan. Interaction time measures external requests and response processing and is reported for the physical experiments. Total time is the accumulated trial duration. Physical result tables and figures use the residual planning-time estimate

$$t_{\text{plan}} = t_{\text{total}} - t_{\text{interaction}} - t_{\text{execution}}.$$

This estimate includes overhead outside the separately recorded interaction and physical execution durations. Direct search time and residual planning time are distinct measurements.

C.8 VLM Prompts

Both domains use the same external VLM configuration. The served model identifier is `nvidia/Gemma-4-31B-IT-NVFP4`. The sampling temperature is 0.3, the maximum completion length is 256 tokens, and the request timeout is 90 seconds. Log probabilities are enabled with the top five token alternatives. Thinking and reasoning output are disabled. A text-only warmup asks whether the model is ready and requests a Yes or No response.

Each experimental request includes two current RGB camera images and the current robot action. Ours supplies a Korean state-verification question, the judgment criterion for the queried predicate, and the domain action model. KnowNo supplies candidate actions with response tokens, judgment criteria for the candidate action types, and the domain action model. TomatoHarvesting additionally supplies an environment model with fixed object identifiers and stem assignments. The following English descriptions summarize the configured prompt instructions. The supplementary configuration files preserve the original English and Korean prompt text.

Shared Request Instructions

For state verification, the VLM answers the supplied Korean question from the two camera images and robot context. The response contains two lines, with Yes or No first and a concise reason second. The current action describes the attempted operation and does not establish that the queried fact is true. The VLM must judge the fact from visible evidence. The client converts Yes to `true` and No to `false` before the belief update.

For action selection, the VLM chooses the most appropriate next action from the supplied candidates. The response contains two lines, with a candidate token first and a concise Korean reason second. The VLM must verify the relevant object, robot state, and execution conditions from the images. The candidate name

and current action do not establish feasibility. If no concrete candidate is appropriate, the VLM selects the option that requests a manually entered action. The client parses the selected token and checks that the token belongs to the supplied candidate set.

WasteSorting Domain Instructions

The action model specifies `detect`, `pick`, `place`, and `done`. Detection requires an empty gripper and identifies visible waste objects and categories. Picking requires a detected object and an empty gripper. Placement requires the robot to hold the selected object and use the bin corresponding to the detected category. Completion requires all waste objects to be sorted and the gripper to be empty. A visible object, bounding box, or category label does not establish that detection is complete. An object exposed after another object is removed must be detected before picking.

State-verification criteria use the identity and appearance of the specified object. For `detected`, the VLM first identifies the specified waste number and checks whether the object is visible on the worktable. Another visible object or the current detection action is insufficient evidence. Occlusion or uncertain identity must not be treated as proof that the object is absent. For category predicates, the VLM checks material and shape. Plastic requires visible features of a plastic bottle or container. A can requires the configured rectangular shape and metal surface, and a round plastic container must not be classified as a can. Paper requires features such as a thin surface, folds, or printed cardboard. The configured general-waste example has a banana shape. Color alone or the bin name in an action candidate is insufficient evidence of category.

For `holding` and `handempty`, the VLM checks the actual contents of the gripper in both images. For `in_bin`, the specified object must be visibly inside the specified bin. For `sorted`, the object category and bin category must match. For `done`, no waste may remain on the worktable, all waste must be in the correct bins, and the gripper must be empty. Waste, robot, and bin-type predicates require visual identification of the specified target. Attempted pick or place actions do not establish the corresponding state facts.

The action-selection criteria require `detect` before `pick` for newly exposed or undetected waste. A pick candidate requires an empty gripper and a visible object with an identifiable number. A place candidate requires the gripper to hold the specified object and the visual category to match the target bin. A completion candidate requires an empty worktable, correct bin placement, and an empty gripper. If visibility, object identity, or execution conditions do not support a concrete candidate, the VLM selects the option for manual action entry.

TomatoHarvesting Domain Instructions

The environment model specifies two tomatoes per stem. Tomatoes 1 and 2 belong to `stem_01`, and tomatoes 3 and 4 belong to `stem_02`. Numbers run from right to left across the scene and remain fixed after occlusion, picking, harvesting, or disposal. The right pair of ArUco markers identifies `stem_01`, and the left pair identifies `stem_02`. The location criterion also states that a view containing only two markers indicates that the robot is at `stem_02`.

The action model specifies navigation, detection, picking, scanning, placement, disposal, and completion. Navigation and detection require an empty gripper. The robot moves to the next stem if no harvestable tomato remains at the current stem. Picking requires an observed ripe tomato at the current stem and an empty gripper. Scanning inspects the held tomato. Placement requires a tomato confirmed fresh after scanning, and disposal requires a tomato confirmed rotten after scanning. Completion requires all necessary tomatoes to be processed. A newly exposed tomato requires detection before picking even if the tomato is visible in the images.

For `at`, the VLM identifies the specified tomato and target stem using the fixed identifiers and markers. Direct visibility at the target stem supports Yes, and direct visibility at the opposite stem supports No. Absence from an image or uncertain identity does not establish that the tomato belongs to another stem. A tomato held by the robot is no longer on the stem, and each stem contains at most two tomatoes. The current action must not be used as location evidence. For `ripe` and `unripe`, the dominant red or green surface color determines ripeness. Scratches and rotten areas are separate from ripeness.

For `fresh` and `rotten`, inspect the surface of the specified tomato actually held in the gripper. Answer No only if a large, distinct beige rotten area is visible. Small scratches, dents, spots, reflections, and minor discoloration near the stalk do not indicate rot and must not be used to reject freshness. Base the judgment on both the scene image and the wrist-camera image.
For `holding`, `holded`, and `handempty`, the VLM checks the actual gripper contents. For `loaded` and `discarded`, the specified tomato must be visibly in the harvest basket or disposal location. The predicates `observed` and `scanned` describe completed procedures. Visible tomatoes and attempted detect or scan actions do not establish procedural completion. For `located`, the VLM checks the robot position relative to the target marker region. Tomato counts use distinct visible tomatoes across both images without double counting or assumptions about occluded regions. Harvested counts use tomatoes visibly inside the basket and do not substitute the number of attempted placements. Tomato, robot, location, and stem predicates require identification from appearance, scene structure, and markers.
The action-selection criteria require the VLM to verify robot location, gripper contents, and the target tomato. Navigation is appropriate if the gripper is empty and movement to another stem is necessary. Detection precedes picking for newly visible or undetected tomatoes at the current stem. Picking requires an empty gripper and a visible ripe tomato at the current stem. Scanning requires the robot to hold the specified tomato and still need a quality judgment. Placement requires the held tomato to appear fresh without a large rotten area. Disposal requires a large, distinct rotten area and must not follow from minor scratches alone. Completion requires no remaining ripe tomatoes to process and an empty gripper. If no concrete action satisfies the visible conditions, the VLM selects the option for manual action entry.

Appendix D Supplementary Evaluation Details

This appendix specifies the threshold protocol and the failure categories used in Sec. 5.2.1. The reported outcomes use the same trial sets as the corresponding main-text comparisons.

D.1 Threshold Protocol

The threshold study evaluates $\tau \in \{0.0, 0.1, \ldots, 1.0\}$ with entropy gain query selection and oracle responses. Each threshold has 200 trials per domain. The planner parameters are those in Appendix C.1, and the shared reference at $\tau = 0.8$ is specified in Appendix C.4. A threshold of zero suppresses queries because belief confidence cannot fall below zero. A threshold of one requires confidence to reach one before the robot continues, unless no differentiating fact remains. Table 6 reports the success rates and successful-trial query counts.

D.2 Ablation Reporting

Table 5 contains Fully-Random, W1–W3, Q1–Q2, and Ours. Each configuration has 200 attempted trials per domain. The success denominator includes the four unparseable W2 outputs. Cost metrics include successful trials only. The policy definitions and domain-specific discount factors are given in Appendix C.3.

D.3 VLM Failure Classification

We classify failed physical trials using the available action candidates, external responses, and subsequent execution. A VLM failure occurs if a response incorrectly verifies a state fact or selects an action that causes failure despite an available feasible alternative. Feasible actions that do not advance the task can also cause a VLM failure if the resulting sequence exhausts the trial limit. A planner failure occurs if the candidate set contains only infeasible actions or the baseline autonomously selects an action whose preconditions are not satisfied. The termination label alone does not determine the category.

Table D3 reports the failure counts. In WasteSorting, Ours has three VLM feedback failures. KnowNo has three VLM failures and 17 planner failures. In TomatoHarvesting, Ours has three VLM feedback

failures. KnowNo has 18 VLM failures and 12 planner failures. Six of the 18 VLM failures involve detection choices that exhaust the step budget despite feasible alternatives for task progress. The remaining 12 involve incorrect action selections.

Table D3: Failure counts in the VLM evaluation. Percentages use failed trials as the denominator.

Domain	Method	Failed trials	VLM failures	Planner failures
WasteSorting	KnowNo	20	3 (15%)	17 (85%)
WasteSorting	Ours	3	3 (100%)	0
TomatoHarvesting	KnowNo	30	18 (60%)	12 (40%)
TomatoHarvesting	Ours	3	3 (100%)	0

D.4 KnowNo Action Requests

KnowNo generates candidate actions from domain information and the current symbolic state. Candidate scores and the calibrated threshold define the conformal prediction set. If the set contains multiple actions, the interface requests one action choice from the external expert. KnowNo requests one response per trigger. The selected action is then executed, and the execution result updates the state information used for the next decision. Appendix C.2 lists the model and calibration parameters.

Appendix E Human Evaluation Procedure

This appendix describes the protocol for the human evaluation in Sec. 5.2.1. It summarizes the participant instructions, trial procedure, and interview guide. The physical platforms and task domains match the VLM evaluation.

E.1 Participant Briefing

The briefing explains the purpose of the study, the sequence of activities, and the participation compensation. Participants complete the research participation and personal information consent forms before the tasks begin. They are informed that participation is voluntary and that they can request a break or withdraw during the session.

Participants receive explanations of the task goals, object categories, available robot actions, and tablet controls for both domains. They then watch example videos of task execution. The instructions ask participants to monitor the camera views and answer robot questions according to their own judgment. Participants are told that an incorrect answer does not interrupt the experiment and that they should select the answer they consider most appropriate if they are unsure.

E.2 Domain Information

In WasteSorting, participants receive information about the four waste categories and the corresponding bins. The available actions detect waste objects, pick an object, and place the held object in a selected bin. In TomatoHarvesting, participants receive information about ripe and unripe tomatoes and the distinction between fresh and rotten tomatoes. The available actions navigate between locations, detect tomatoes at a stem, pick a tomato, scan a held tomato, and place or discard the tomato. The task goal requires fresh ripe tomatoes to be harvested, rotten tomatoes to be discarded, and unripe tomatoes to remain on the stem.

Each WasteSorting trial contains four objects. Participants are taught to distinguish general waste, paper, plastic, and cans and to place each object in the corresponding bin. The interface marks the categories with blue, green, yellow, and gray boxes, respectively. Object identifiers follow detection order and a position rule if several objects are detected together. Participants are instructed that the identifiers do not indicate object categories.

Each TomatoHarvesting trial contains four tomatoes distributed across two stems. The briefing distinguishes three appearances corresponding to fresh ripe tomatoes, rotten ripe tomatoes, and fresh unripe tomatoes. Participants are taught that ripeness can be judged from a distance, whereas a closer inspection is needed to judge rot. The interface marks fresh ripe tomatoes in green, unripe tomatoes in yellow, and rotten tomatoes in red. The robot can detect and pick tomatoes only at its current stem. Tomato identifiers follow detection order, with the rightmost tomato assigned first if several tomatoes are detected together. The task ends after the tomatoes have been processed according to the task rules and the robot has returned to the charging station.

E.3 Tablet Interaction

The tablet interface displays robot camera views and the current query. Ours presents a question about a selected state fact with Boolean response options. KnowNo presents candidate actions and requests an action choice. Participants choose a response through the tablet, and the robot continues execution after the response is submitted. Ours can request several state confirmations before the confidence threshold is met. KnowNo requests one action choice per trigger.

Participants start each task through the corresponding start button and use the response buttons on the right side of the screen to answer queries. Voice, gesture, and manual control functions are not used in the experiment. If a communication problem prevents interaction, execution is paused until the connection is restored.

E.4 Trial Schedule

The planned evaluation comprises five participants with two trials per domain and method for each participant. This schedule yields 10 trials per domain and method and 40 trials in total. Table 4 gives the assigned domain and method orders. Three participants start with WasteSorting and two with TomatoHarvesting. In each domain, three participants start with Ours and two with KnowNo.

Each participant completes four consecutive trials in one domain before the four trials in the other domain. The assigned sequences vary the domain and method orders across participants. With five participants, the orders are not fully balanced.

E.5 Trial Procedure

A facilitator explains the tasks and manages the session, while an assistant prepares the robot and object arrangement for each assigned scenario. Before each trial, the experimenters reset the environment, check the selected method and scenario, and confirm that the camera views are available. The facilitator starts the tablet screen recording, and the experimenters check that the trial logs are saved after execution.

After each trial, the participant completes a questionnaire about the robot system just experienced. The questionnaire contains nine items about system understanding and performance, with responses on a seven point scale from strong disagreement to strong agreement. Participants are asked to evaluate the most recent trial rather than combine experiences from earlier trials. They can rest while the assistant prepares the next scenario. This sequence is repeated for all eight trials, followed by an interview about the complete experience.

At the end of the session, the experimenters save the tablet and external camera recordings and verify that the logs match the participant identifier, trial order, domain, and method. These records document task execution and the responses supplied through the interface.

E.6 Interview Guide

The interview guide invites participants to describe both favorable and unfavorable experiences with the two methods. Participants can report no difference or state that they do not remember a particular event. The interviewer asks all six core questions and selects additional prompts only if clarification or a concrete example is needed. The questions below are English translations of the Korean guide.

1. What was the most memorable difference between the two methods? Please also say if you noticed no difference.
2. For each method, what did you look at and think about before you chose an answer to a robot question? Please describe a question that you remember.
3. Did the information on the screen or the response options help you choose an answer or make it difficult?
4. How did you feel about the timing of the robot questions? Please describe a scene that you remember.
5. Across the entire task, how did the two methods differ in the effort you spent and your sense of task progress?
6. Which method would you choose for a similar task, and why? You may have no preference or choose differently depending on the situation.

Additional prompts ask what participants checked before an answer, whether they considered task rules or subsequent robot actions, and whether they hesitated. Prompts about the interface ask whether the target object was clear, whether the camera views provided enough information, and whether the available options included the intended answer. Prompts about timing ask about opportunities to intervene, repeated questions, and interruptions to observation or thought. Prompts about effort distinguish the difficulty of an individual judgment from the effort of repeated responses and continued monitoring. The final prompts ask whether preferences differ across domains and which improvement participants would prioritize.

If further explanation is needed, the guide permits one or two common examples of questions from the two methods. The interviewer explicitly identifies these as prepared examples that may differ from the actual task experience. Participants describe the information and judgments needed to answer each example and indicate whether they recall a similar event during the experiment. The interviewer ends with a summary of the main responses and invites corrections or further comments.